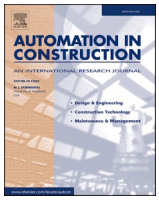
Automation in Construction 160 (2024) 105293



Contents lists available at ScienceDirect

Automation in Construction

journal homepage: www.elsevier.com/locate/autcon





Enhancement and validation of ifcOWL ontology based on Shapes Constraint Language (SHACL)

Chaoyue Wang [a], Liang Zhang [b,*], Wei Yan [a,*]

[a] School of Information Science and Engineering, Shandong Normal University, Jinan 250399, China
[b] Institute of Frontier and Interdisciplinary Science and Key Laboratory of Particle Physics and Particle Irradiation (MOE), Shandong University, Qingdao 266237, China

ARTICLE INFO

Keywords:
Industry foundation classes (IFC)
Shapes constraint language (SHACL)
IFC-OWL conversion
Web ontology language (OWL)
EXPRESS rules

ABSTRACT

In the domain of Building Information Modeling (BIM), although ifcOWL ontology has already achieved a fundamental conversion from the Industry Foundation Classes (IFC) schema to the Web Ontology Language (OWL) format, deeper research is still needed to address complex attributes and constraints in the IFC schema. This paper aims to achieve the conversion and validation of *WHERE* rules used for defining data integrity constraints in the IFC schema. A method is introduced, which entails converting *WHERE* rules into SPARQL-based Shapes Constraint Language (SHACL) constraints. The application of SHACL constraints for validating the ifcOWL instance graph enables the effective identification and resolution of data errors and inconsistencies related to *WHERE* rules. The implementation of comprehensive validation and quality control mechanisms guarantees the reliability and accuracy of ifcOWL instance graphs. Furthermore, high-quality data supports subsequent research related to ifcOWL, such as quality control and cross-domain interoperability.

1. Inroduction

The exchange and interoperability of data between different domains has long been an issue of great concern. Despite the significant advantages of Building Information Modeling (BIM) in architectural design, construction, and operations management, its interoperability issues are still considered to be the major obstacle that restricts its further development and promotion [1–5]. To meet the urgent need for a unified, comprehensive, standardized and extensible building data model in the Architecture, Engineering and Construction (AEC) domain, buildingSMART [6] developed the Industry Foundation Classes (IFC) schema, aiming to improve the interoperability and information exchange between various software, tools and stakeholders in this field through standardized data structures and semantics. It is a neutral Computer-Aided Design (CAD) data exchange data schema that is independent of various BIM software system vendors and supports all phases of the construction project lifecycle. Since being adopted by the International Organization for Standardization (ISO) [7] in 2013, the IFC schema specification has become the basis for implementing open BIM in many countries [8].

In recent years, with the introduction of concepts such as smart cities, digital twins, and smart buildings, many studies have been dedicated to achieving the integration and interoperability of the IFC schema with other domains like Geographic Information Systems (GIS) [4,9,10], Internet of Things (IoT) [11–13], and Facility Management (FM) [5,14,15]. This endeavor aims to facilitate more effective multi-domain collaborative management and achieve greater overall benefits. However, the significant differences in data structures and semantics across various domains undoubtedly pose numerous challenges to cross-domain data exchange. In this context, Semantic Web technology has played an important role. As a concept proposed by World Wide Web Consortium (W3C), Semantic Web technology aims to enrich the semantics of data on the internet, enabling machines to understand and process web information [16]. This technology facilitates the effective management, linking, and sharing of knowledge across different systems and applications. Additionally, it provides powerful tools for integrating data from multiple heterogeneous sources, supporting complex cross-domain search queries and inference [17–19]. Semantic Web utilizes ontology to formalize modeling. An ontology is typically defined using the Web Ontology Language (OWL) [20,21] to define concepts, properties, and relationships. Therefore, improving the semantic clarity of IFC through ontology is considered a feasible approach to enhance IFC interoperability [22–25]. The buildSMART organization has published the widely adopted IFC ontology, known as ifcOWL, based on the

* Corresponding authors.
E-mail addresses: zhang.l@sdu.edu.cn (L. Zhang), wyan@sdnu.edu.cn (W. Yan).

https://doi.org/10.1016/j.autcon.2024.105293
Received 24 April 2023; Received in revised form 20 January 2024; Accepted 21 January 2024
Available online 31 January 2024
0926-5805/© 2024 Elsevier B.V. All rights reserved.

conversion patterns from EXPRESS to OWL summarized by Pauwels et al. [26]. Currently, ifcOWL has been widely applied in the field of BIM to promote integration and interoperability with other domains. Additionally, it provides a structured data model and semantics for distributed data management, while also supporting semantic search and reasoning capabilities [19,22,27–29].

We show in detail in Table 1 the conversion patterns of the IFC schema from EXPRESS to OWL as summarized by Pauwels et al. We can see that Pauwels et al. do not address the conversion of *DERIVE* attributes and *WHERE* rules from EXPRESS to OWL. Both the *WHERE* rule and the *DERIVE* attribute depend directly on the content and structure of other attributes, so the conversion of both should be paired with additional specific rules or constraints that constrain their values to the attributes on which they depend to avoid inconsistencies [26]. Since the main application scenario discussed in the literature is the conversion of IFC STEP files generated by IFC-compliant software tools to ifcOWL instance graphs, they have overlooked the conversion of *DERIVE* attributes and *WHERE* rules [26]. Indeed, the *WHERE* rules play a crucial role in IFC schema. They impose stringent constraints on the entities and types within the IFC schema, ensuring both logical consistency and data accuracy. However, in the current conversion scheme from IFC schema to ifcOWL, these rules have not been adequately addressed. The lack of effective conversion for *WHERE* rules means that the data quality and reliability of ifcOWL cannot be assured, inevitably increasing the risk of data errors and inconsistencies.

Table 1
Conversion pattern of the IFC schema from EXPRESS to OWL as summarized by Pauwels et al.

EXPRESS		ifcOWL
Schema		Ontology
Simple data type		owl:Class and owl:DatatypeProperty restriction
Defined data type		owl:Class
Aggregation data type	LIST	The OWL class express:List represents list elements. Each element of the list is related to the following elements in the list by using the object property express: hasNext. Each list element is linked with its actual content via the object property express:hasContents.
	ARRAY	Same as LIST, adding a restriction to set the size of the array.
	SET	The common non-functional object property in OWL. This property can be assigned to the same instance an unlimited number of times. Cardinality restrictions for this non-functional property allow to represent possible constraints on the size of the set.
Constructed data type	ENUMERATION data type	The ENUMERATION data type is converted to the owl:Class, which is equivalent to one of a limited set of named individuals (owl:NamedIndividual), each individual representing one enumeration item.
	SELECT data type	The SELECT data type is converted to the owl:Class, which is equivalent to the union of a limited set of owl:Class statements.
Entity data type	Class hierarchy	The entity data type is converted to the owl:Class. The SUBTYPE OF declaration is converted to the rdfs:subClassOf.
	Regular attribute	The object property declaration in OWL with the addition of a proper set of restrictions to the declaration of the owl: Class.
	DERIVE attribute	–
	WHERE rule	–
	The OPTIONAL keyword	owl:maxQualifiedCardinality
	INVERSE attribute	The object property declaration in OWL with the owl:inverseOf.
FUNCTION and RULE		

Currently, the research on integration and interoperability between BIM and other domains, mainly includes three information flow patterns: from BIM to other domains, from other domains to BIM, and from BIM and other domains to third-party systems [9]. In the discussion of information flow pattern from BIM to other domains, a widely researched subject is the conversion of IFC files into CityGML files for the GIS field, to support urban planning and management. In this process, the ontological method is widely applied in related studies [9]. The core of this method is to convert the IFC schema into the ifcOWL ontology and establish mappings with ontologies from other domains. Mature tools like IFCtoRDF [30] and IfcSTEP-to-IfcOWL Converter [31] provide effective technical support for the conversion of IFC files to ifcOWL instance graphs in existing research and practice. However, the process of data conversion may introduce errors when dealing with complex data structures or large datasets. Furthermore, the dynamic and iterative nature of the engineering environment can also lead to errors in the ifcOWL instance graph as it may be continuously modified or extended during the course of the project. In the aforementioned scenarios, the *WHERE* rules need to be used as an additional validation layer to ensure that no relevant errors are introduced during the data conversion process, as well as in the modification and extension of the ifcOWL instance graph. In research on the information flow pattern from other domains to BIM, researchers primarily aim to convert non-BIM source data into IFC data, which can then be imported or integrated into BIM models. For example, recent studies have explored the conversion of specific data from CityGML files, within the GIS domain, into IFC format [32,33]. The aim is to enrich BIM models with geospatial background data, thereby addressing various design-related issues. In this type of information flow pattern, ontology-based methods typically require data mapping of non-BIM domain sources to create a new ifcOWL instance graph. Therefore, when using data from other domains to support the BIM model, it is necessary to ensure that the ifcOWL instance graph created from scratch complies with the original IFC schema, including adhering to the constraints defined in the *WHERE* rules of the IFC schema. Pauwels et al. [26] similarly emphasized that if the objective is to instantiate an Resource Description Framework (RDF) [34] graph from scratch while adhering to the original IFC schema, it is necessary to add *WHERE* rules in the ifcOWL ontology to check the consistency of the graph. In summary, in the context of multi-domain data integration and interoperability, ifcOWL serves as an intermediary for data mapping, and its accuracy and completeness have an impact on the quality and reliability of data. Therefore, using *WHERE* rules to validate the ifcOWL instance graph during the data conversion process is an important measure to effectively improve data quality and consistency in the integration and interoperability process.

Due to the adoption of the Open-World Assumption (OWA) and the lack of the unique name assumption, it is difficult for OWL to define the structural constraints of RDF graphs [35]. Currently, how to check the correctness and completeness of data sets is still a problem that needs to be solved in RDF-based applications [36]. Therefore, the *WHERE* rules, which are essential for ensuring data consistency and integrity in the IFC schema, have not been included in ifcOWL. This shortcoming in the conversion restricts the practicality and reliability of the data in the new environment to some extent because the *WHERE* rules do not retain their original level of constraints in ifcOWL. The Shapes Constraint Language (SHACL) [37] was created as an option to solve the above problems. SHACL is a data modeling language for formalizing the constraints needed in the RDF data validation process [38]. The language consists of two components, SHACL Core and SHACL-SPARQL. The former is used to represent the basic syntax and structure of shapes, constraints and targets, and the latter is required for SPARQL-based constraints and other extension mechanisms [39]. In SHACL, validation is based on shapes, which define specific constraints and indicate which nodes in the RDF graph should be validated against these constraints [40]. In general, SHACL shapes can be categorized into two types based on their

target: node shapes (sh:NodeShape) which apply constraints to node targets, and property shapes (sh:PropertyShape) which apply constraints to property targets [41]. The SHACL validation process is shown in Fig. 1. This process takes the data graph and shapes graph as input, where the data graph contains the RDF data that needs validation, and the shapes graph consists of a set of shapes with each shape defining constraint conditions for the target nodes or properties in the data graph [37,38,42]. When validating RDF data, each shape in the shapes graph selects the focus nodes to be validated from the data graph through target declarations. Furthermore, SHACL provides mechanisms to filter and process the focus nodes, allowing validation to be performed only on nodes that meet specific criteria. Finally, the focus nodes that satisfy the filtering conditions will be validated according to the rules defined in the shape definition. The SHACL processor generates a validation report for the user to analyze and handle, which contains detailed error or warning messages about the violations. When there are no errors reported, the RDF graph is considered to have passed validation [43].

The similarities between SHACL and OWL are that they are both based on the RDF model and officially recommended by the W3C. However, unlike OWL, SHACL is a language based on the Closed World Assumption (CWA), which allows adding constraints and limitations to triples in an RDF graph through the definition of shapes for consistency checking [37]. SHACL validation can be used to detect problems in a dataset, and it can also provide data quality guarantees for the purpose of data exchange and interoperability [40]. Many modern triple stores support SHACL validation, including GraphDB, Apache Jena, and Neo4j [41]. In addition, some ontology editing software, such as TopBraid Composer, has the ability to be compatible with OWL and SHACL, and also enables automatic conversion from Resource Description Framework Schema (RDFS) and OWL to SHACL. Therefore, combining SHACL and OWL can solve the problem of lack of constraint descriptions in the data graph and help us build a more complete data system.

To improve the quality and consistency of data in cross-domain data integration and interoperability processes, we focus on *WHERE* rules in the IFC schema and propose an approach to enhance ifcOWL ontology based on SHACL. This method adds *WHERE* rules to the ifcOWL ontology through SPARQL-based SHACL constraints, thus the data in the ifcOWL instance graph can be validated according to the *WHERE* rules defined within the entities and types in the original IFC schema. The paper is organized as follows: Section 2 reviews a series of improvements so far for ifcOWL and related studies using SHACL within the AEC domain. A brief introduction to type declaration and entity declaration in the EXPRESS language is given in Section 3. Section 4 describes in detail how to convert built-in constant, operators and built-in functions in the EXPRESS language to SPARQL constraints against *WHERE* rules. Section 5 introduces the conversion pattern of *WHERE* rules to SPARQL-based SHACL constraints. In Section 6, we demonstrated how our developed program automatically generates SHACL constraints for *WHERE* rules and applies them to validate data in ifcOWL instance graphs. In Section 7, we evaluate and discuss the proposed conversion pattern of the study. Section 8 summarizes our current research and describes future work to be undertaken.

2. Related work

2.1. Enhancements and improvements to the ifcOWL ontology

The multi-stakeholder context of the construction industry has created problems such as poor information interoperability and low productivity, which stem from difficulties in information reuse [24]. Therefore, many researchers have attempted to address these problems through the use of ontologies. A recommended and complete version of IFC ontology would be beneficial to ease the integration of data models, support the flow of data and enable the use of ontology-based tools [44]. Many example applications have emerged for IFC to OWL conversion, covering a wide range of application scenarios in the building industry, smart city construction, facility management, factory and manufacturing areas [26]. There is no doubt that enhancements and improvements to the ifcOWL ontology will bring greater potential and more opportunities to the AEC domain, hence a number of researchers still maintain a strong interest in it.

Since the first unidirectional conversion pattern from IFC to OWL was designed by Schevers et al. [45], the improvement of IFC ontology has continued till now. After studying previous work on conversion from the IFC EXPRESS schema to the ifcOWL ontology, Pauwels et al. [26] summarized a set of criteria to be followed towards a usable and recommendable ifcOWL ontology and a more complete conversion pattern from IFC to OWL. They also developed a tool IFCtoRDF [30] for parsing IFC STEP physical files and converting them into RDF graphs. In addition, Pauwels et al. [46] proposed two main improvements on the basis of ifcOWL. First, they proposed to modularize the ontology and divide the IFC ontology into different and separate modules [46]. Secondly, four representations are given in order to represent the list data type in IFC as a simpler semantic structure in OWL [46]. The above two initiatives effectively reduce the size and complexity of ifcOWL instance graphs. Guo et al. [47] pointed out that ifcOWL only uses simple pre-existing WKT expressions (point or linestring) to describe BIM geometric aggregated data. To address this issue, they proposed a method to represent BIM geometric data in ifcOWL ontology through WKT high order expressions. This representation not only allows for the utilization of existing WKT expressions to generate a more concise RDF representation but also reduces the loss of information [47].

Previous research has made improvements to the ifcOWL ontology at different levels and has made some progress. However, it is worth noting

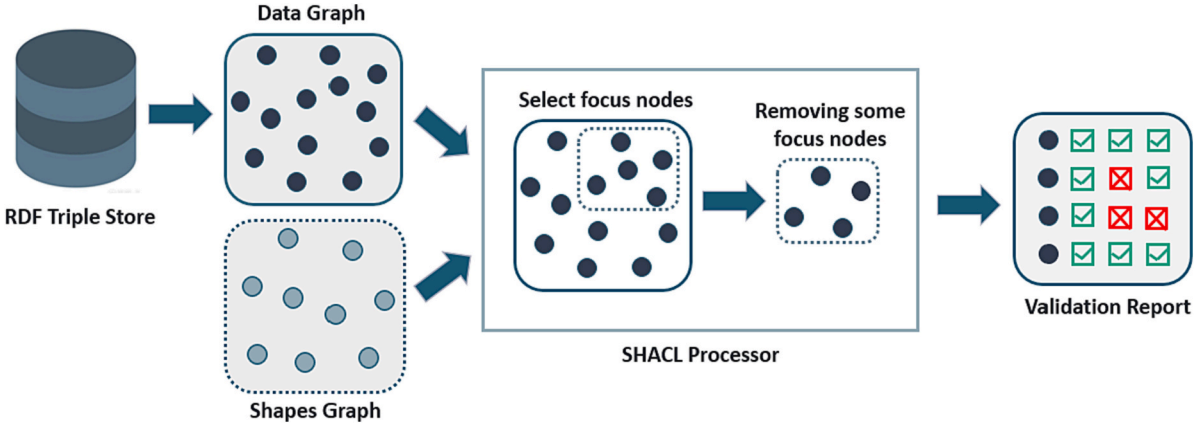


Fig. 1. SHACL validation process.

that the *WHERE* rules in the IFC schema have not yet been effectively converted. In EXPRESS, *WHERE* rules are used as part of global rules to further constrain entity data types or defined data types. Therefore, the conversion of *WHERE* rules requires not only adding constraints to the ifcOWL instance graph but also performing constraint validation on the ifcOWL instance graph. Due to the complexity of the semantics and syntax of *WHERE* rules, as well as their mismatch with OWL semantics, there is currently no research that can provide an effective and comprehensive conversion for them.

There have been some preliminary efforts in converting the *WHERE* rules. In the mapping method of EXPRESS to OWL/SWRL proposed by Zhao and Liu [48], they argue that constraints in *WHERE* rules can be mapped to SWRL rules. However, since SWRL was submitted to the W3C in 2004, it is still not a W3C-recommended language, making it difficult to extend its scope to the industry [49]. Many researchers have also pointed out the limitations of SWRL in industrial applications based on ontology modeling. First, it is based on OWA, whereas CWA-based query and inference techniques are required in applications with known boundaries and for validating data [50]. Additionally, SWRL exhibits high complexity in managing rules and updating information [49]. Terkaj et al. [44] proposed a method to further optimize the ifcOWL ontology through class expressions. However, this method only involves the conversion process of *WHERE* rules in some entities related to entity *IfcRelDefinesByType* and entity *IfcPreDefinedPropertySet*. Moreover, their proposed method partially modifies the recommended conversion standard to enhance the ifcOWL ontology, which makes the original conversion standard more complex, difficult to maintain, and increases the risk of introducing errors and compatibility issues. In conclusion, there is still a lack of an ideal solution for the conversion of *WHERE* rules at present.

2.2. Application of SHACL in AEC domain

Since SHACL became an official W3C recommended standard in 2017, an increasing number of researchers have started introducing SHACL into the AEC domain for BIM model validation. Kücükavci et al. [51] proposed a method for automating BIM validation using SHACL shapes and the FSO/FPO ontology. This approach ensures that the BIM model includes the HVAC information required for simulating hydraulic systems. Kovacs et al. [52] discusses a BIM Quality Control Ecosystem that is based on Requirement Linked Data in order to create a framework where automated BIM compliance checking methods can be widely used. The study primarily utilizes SHACL for automatically generating compliance reports for BIM. In computer-based damage analysis of buildings, Hamdan et al. [53] performed knowledge-based damage classification and assessment of buildings by OWL and SHACL. In addition, they noted that SHACL allows rules to be defined based on CWA without the restrictions of description logic, so SHACL has stronger expressive power when defining rules compared to OWL [53]. Werbrouck et al. [54] concluded that combining OWL and SHACL could form a powerful tandem at different inspection stages of a distributed building dataset. In the first stage, the OWL reasoning mechanism inferred the implicit statements, while in the next stage SHACL can be used to validate the model [54]. As stated above, SHACL can add CWA-based enhancements and supplements to the OWA-based ontology tool or application, and accordingly will help us build a more complete information system.

In the field of ifcOWL application, some studies have also begun to experiment and apply SHACL as a powerful tool for data validation and integrity checks. Hagedorn et al. [55] present a method for semi-automated validation of linked data sets of building models, using a tunnel construction project as an example. In the data preparation phase, they convert the building model from IFC EXPRESS schema to ifcOWL. They then use SHACL to validate whether the technical rules defined within model instances are satisfied. Senthilvel et al. [56] proposed a visual programming interface for creating SHACL shapes to support the ifcOWL ontology and use cases, and also demonstrated the feasibility of using SHACL to implement linked building data checking. Soman et al. [42] proposed a Linked-Data based Constraint-Checking (LDCC) approach to model and validate complex dynamic scheduling constraints in the construction process. The approach uses ifcOWL to capture model information along with a customized ontology, LinkOnt, to capture resource information and links, and SHACL to model construction scheduling constraints to detect violations [42].

In the aforementioned studies, researchers defined different SHACL rules for validating building models. The specific rules and constraints varied depending on the application scenarios. By combining SHACL with ifcOWL, it becomes possible to leverage SHACL rules to validate and analyze building models in the OWL format, ensuring data integrity and accuracy. This combination fully demonstrates the application potential of SHACL in the field of construction engineering. Based on the constraint definition and validation capabilities of SHACL, we propose a method that uses SPARQL-based SHACL constraints to convert and validate *WHERE* rules. The aim of this method is to address the current lack of standardized approaches for converting and validating *WHERE* rules. By applying this method, it is possible to improve the quality and consistency of building information during cross-domain data integration and interoperability.

3. Defined data type declarations and entity data type declarations

Although the IFC was developed independently of the ISO standardization body and the STEP procedure, it shares many of the same underlying technologies, most notably the data modeling language EXPRESS, as defined in Part 11 of the STEP standard [57]. EXPRESS is an object-oriented information model description language that provides the detailed description of the specification (or definition) of the objects involved in the domain of the argument, as well as the description of the information and constraints associated with these objects [58]. In EXPRESS, entity data types are established by entity declarations and defined data types are described by type declarations. For types and entities with *WHERE* rules in IFC schema, we will introduce the EXPRESS declaration of both in detail in this section.

3.1. Type declaration

The type declaration is used to establish a defined data type and give a reference identifier that starts with the keyword *TYPE* and ends with the keyword *END_TYPE* [58]. The defined data type adds a constraint to the definition domain via the *WHERE* rule, which is an expression that returns a logical value that must be TRUE [58]. As an example, Fig. 2 shows the EXPRESS specification of the IFC type *IfcPositiveLengthMeasure*, where the *WHERE* rule *WR1* constrains that the positive measure should be greater than zero [59]. This constraint applies to all entities that have type *IfcPositiveLengthMeasure* as the data type of the attribute. For example, Fig. 3 gives the EXPRESS specification of the IFC entity *IfcWindow*, where the explicit attributes *OverallHeight* and *OverallWidth* have the data type *IfcPositiveLengthMeasure*, hence each instance of this entity should be assigned a value to these two attributes respectively, and the value needs to satisfy the *WHERE* rule *WR1*.

```
TYPE IfcPositiveLengthMeasure = IfcLengthMeasure;
WHERE
WR1 : SELF > 0.;
END_TYPE;
```

Fig. 2. EXPRESS specification of the IFC type *IfcPositiveLengthMeasure*.

```
ENTITY IfcWindow
SUBTYPE OF (IfcBuiltElement);
OverallHeight : OPTIONAL IfcPositiveLengthMeasure;
OverallWidth : OPTIONAL IfcPositiveLengthMeasure;
PredefinedType : OPTIONAL IfcWindowTypeEnum;
PartitioningType : OPTIONAL IfcWindowTypePartitioningEnum;
UserDefinedPartitioningType : OPTIONAL IfcLabel;
WHERE
CorrectPredefinedType : NOT(EXISTS(PredefinedType)) OR(PredefinedType <>
IfcWindowTypeEnum.USERDEFINED) OR((PredefinedType =
IfcWindowTypeEnum.USERDEFINED) AND EXISTS (SELF\IfcObject.ObjectType));
CorrectTypeAssigned : (SIZEOF(IsTypedBy) = 0) OR('IFC4X3.IFCWINDOWTYPE' IN
TYPEOF(SELF\IfcObject.IsTypedBy[1].RelatingType));
END_ENTITY;
```

Fig. 3. EXPRESS specification of the IFC entity *IfcWindow*.

3.2. Entity declaration

In the EXPRESS language, an entity declaration defines an entity data type and specifies the identifier that references it. The entity declaration not only contains the description of all attributes and related rules of the entity, but also includes the inheritance hierarchy relationship between the entity and other entities [58].

The attributes in the entity declaration are used to represent the characteristics and features of the entity, which are classified as explicit, derived and inverse attributes [58]. Explicit attributes take values that are given directly, and such attributes are represented by values of other data types. Derived attributes are declared with the keyword *DERIVE*, and their values are calculated by expressions. The declaration of the inverse attribute follows the keyword *INVERSE*, which is used to indicate the relationship of the declared entity to an attribute in another entity. In addition, the entity definition also includes the uniqueness rule and the *WHERE* rule for adding further constraints, which are declared by the keyword *UNIQUE* and the keyword *WHERE*, respectively. The *WHERE* rule specifies a constraint on the value taken by a single attribute or combination of attributes for each entity instance [58]. Similar to the type declaration, the *WHERE* rule within the entity declaration is an expression that evaluates to TRUE; otherwise, the population of the EXPRESS schema is considered invalid [60]. The EXPRESS specification of the IFC entity *IfcCartesianPoint* is given as an example in Fig. 4. In the IFC schema, this entity represents a point defined by coordinates in a Cartesian coordinate system [59]. This EXPRESS specification contains the definition of the entity name, the superclass of this entity, the explicit attribute, the derived attribute and the *WHERE* rule. The *WHERE* rule named *CP2Dor3D* constrains that the dimension of each instance of this entity must be 2 or 3.

```
ENTITY IfcCartesianPoint
SUBTYPE OF (IfcPoint);
Coordinates : LIST [1:3] OF IfcLengthMeasure;
DERIVE
Dim : IfcDimensionCount := HIINDEX(Coordinates);
WHERE
CP2Dor3D : HIINDEX(Coordinates) >= 2;
END_ENTITY;
```

Fig. 4. EXPRESS specification of the IFC entity *IfcCartesianPoint*.

4. The conversion pattern of *WHERE* rules

Considering that SHACL can describe and validate complex constraints of RDF graphs, we propose to convert the constraints defined in *WHERE* rules to SPARQL-based SHACL constraints. The constraints will be linked to the ifcOWL target classes and used to validate that the RDF data in the ifcOWL instance graph satisfies the *WHERE* rules in the original IFC schema.

In EXPRESS, the expression in a *WHERE* rule is composed of operators, operands and function calls. Therefore, the conversion of *WHERE* rules should first clarify how to convert operators, functions, etc. from EXPRESS to SPARQL statements. In this section, we describe in detail the conversion pattern from built-in constant, operators and built-in functions in EXPRESS to SPARQL statements, and enumerate some types and entities in IFC 4.3.0.0 [59] as examples.

4.1. Conversion of built-in constant SELF

In EXPRESS, the built-in constant *SELF* in the entity declaration refers to the instance of the entity, while *SELF* in the type declaration represents the type value. According to the conversion pattern in Table 1, both entity instances and type values are converted to individuals of the corresponding classes in OWL. Therefore, referring to the instance of an entity by *SELF* in EXPRESS is equivalent to querying the individual of the corresponding OWL class in the ifcOWL instance graph, while referencing a type value in EXPRESS is equivalent to querying the individual of the corresponding OWL class in the ifcOWL instance graph, and then querying the literal value of the specified datatype property linked to the individual.

For example, the semantic definition of the IFC type *IfcPositiveLengthMeasure* is a length measure greater than zero, and its EXPRESS specification is given in Fig. 2. There is a *WHERE* rule named *WR1* in the type declaration specifying that the positive measure should be greater than zero, where the *SELF* points to the value of type *IfcPositiveLengthMeasure*. The associated RDF graph for the OWL class *IfcPositiveLengthMeasure* is shown in Fig. 5. The defined data type *IfcPositiveLengthMeasure* in the IFC schema is converted to the OWL class *IfcPositiveLengthMeasure*. For all entities in the IFC schema that use this data type as the attribute data type, each entity instance is assigned an attribute value that conforms to this data type, and this value is converted to an individual of the OWL class *IfcPositiveLengthMeasure*, e.g. *IfcPositiveLengthMeasure_14698*. Finally, each individual is linked to its actual content via the datatype property *express:hasDouble*. Therefore, referring to the value of IFC type *IfcPositiveLengthMeasure* using *SELF*, in the ifcOWL instance graph, is equivalent to querying the individual of the OWL class *IfcPositiveLengthMeasure,* and then querying the literal value of the datatype property *express:hasDouble* linked with the

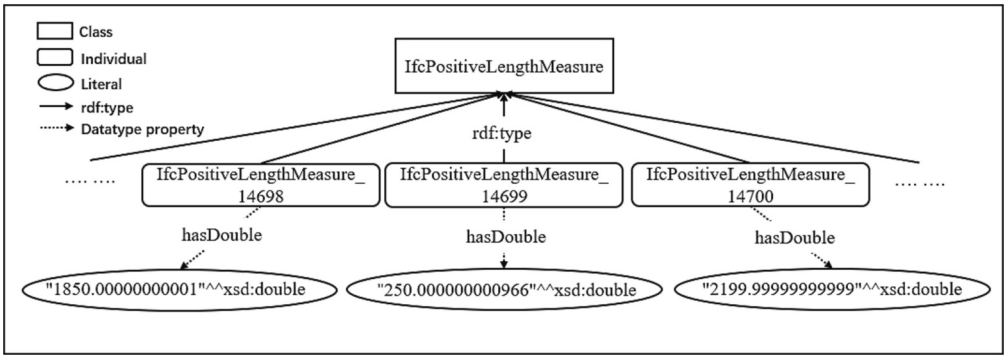


Fig. 5. Associated RDF graph for the OWL class *IfcPositiveLengthMeasure*.

individual.

4.2. Conversion of operators

4.2.1. Arithmetic operators

In the *WHERE* rule expressions of the IFC schema, commonly used arithmetic operators include addition (+), subtraction (−), multiplication (*), division (/), and modulo operation (*MOD*). Take the EXPRESS specification of the entity *IfcUShapeProfileDef* shown in Fig. 6 as an example, which defines a section profile that provides the defining parameters of a U-shape (channel) section to be used by the swept area solid [59]. The *WHERE* rule *ValidFlangeThickness* requires that the value of the attribute *FlangeThickness* should be less than half the value of the attribute *Depth* for each instance of this entity, and the expression of *WHERE* rule uses the division sign (/) for arithmetic operation.

The above four arithmetic operators correspond to the same arithmetic operators in SPARQL, but there is no operator corresponding to the *MOD* in SPARQL. Therefore, we propose to implement the modulo operation according to eq. (1), where *div* refers to the integer division operator. In SPARQL, despite the absence of a dedicated operator for integer division, integer division can still be implemented by passing the result of *A/B* to the *FLOOR* function. This method is feasible because, in SPARQL, the *FLOOR* function is designed to round down any given numerical value to its nearest integer.

$$A \bmod B = A - (A \ \ div \ \ B) \times B \tag{1}$$

4.2.2. Relational operators

4.2.2.1. Value comparison operators. The value comparison operators in EXPRESS can be applied to numeric, logical, string, and other operands

```
ENTITY IfcUShapeProfileDef
SUBTYPE OF (IfcParameterizedProfileDef);
Depth : IfcPositiveLengthMeasure;
FlangeWidth : IfcPositiveLengthMeasure;
WebThickness : IfcPositiveLengthMeasure;
FlangeThickness : IfcPositiveLengthMeasure;
FilletRadius : OPTIONAL IfcNonNegativeLengthMeasure;
EdgeRadius : OPTIONAL IfcNonNegativeLengthMeasure;
FlangeSlope : OPTIONAL IfcPlaneAngleMeasure;
WHERE
ValidFlangeThickness : FlangeThickness < (Depth / 2.);
ValidWebThickness : WebThickness < FlangeWidth;
END_ENTITY;
```

Fig. 6. EXPRESS specification of the IFC entity *IfcUShapeProfileDef*.

[58]. As shown in Fig. 3, in the EXPRESS specification of entity *IfcWindow*, the *WHERE* rule *CorrectPredefinedType* requires either that the attribute *PredefinedType* not be set or that the inherited attribute *ObjectType* be provided if *PredefinedType* is set to *USERDEFINED* [59]. This *WHERE* rule expression uses the value comparison operators equal sign (=) and unequal sign (<>), and we can easily convert these operators to SPARQL operators. Table 2 lists the conversion pattern between the EXPRESS value comparison operator and the SPARQL operator.

4.2.2.2. Membership operators. The membership operator *IN* is used to check whether a member item belongs to an aggregation [58]. As an example, the EXPRESS specification of type *IfcTextAlignment* is shown in Fig. 7, which describes how the text is aligned within the element. The membership operator *IN* is used within the *WHERE* rule *WR1*, and the right-hand side of this operator lists the allowed values for text alignment, and the result of the operation of the expression is TRUE if the type value is equal to one of the allowed values in the aggregation. There is a function *IN* in SPARQL with a similar function to the EXPRESS membership operator *IN*, which is used to check whether the RDF term on the left-hand side is found in the values of list of expressions on the right-hand side [61]. Therefore, we propose to convert the membership operator *IN* to the SPARQL function *IN*.

4.2.3. Logical operators

The EXPRESS logical operators that appear in the IFC schema include *AND*, *OR*, *NOT*, and *XOR*, each of which produces a logical result. For example, Fig. 8 shows the EXPRESS specification of the entity *IfcAxis2Placement3D*, which provides the location and orientation to place items in a three-dimensional space [59]. The *WHERE* rule *AxisAndRefDirProvision* in the entity declaration requires that either both, *Axis* and *RefDirection* are not given and therefore defaulted, or both shall be given [59]. The logical operators *NOT* and *XOR* are used in the *WHERE* rule expression. The logical operators that return boolean values also exist in SPARQL, including the unary operator "!" and the binary operators "||" and "&&", but there is no operator similar to the EXPRESS logical operator *XOR* in SPARQL, we can realize the function of this operator by other logical operators according to Eq. (2). Therefore, the correspondence between EXPRESS logical operator and SPARQL logical operator is shown in Table 3, where *A* and *B* in EXPRESS represent the

Table 2
Conversion pattern of value comparison operators.

EXPRESS	SPARQL
=	[c]=
<>	[c]!=
>	[c]>
<	[c]<
≥	[c]≥
≤	[c]≤

```
TYPE IfcTextAlignment = STRING;
WHERE
WR1 : SELF IN ['left', 'right', 'center', 'justify'];
END_TYPE;
```

Fig. 7. EXPRESS specification of the IFC type *IfcTextAlignment*.

sub-expressions on both sides of the logical operators, and *A* and *B* in SPARQL represent the SPARQL statements corresponding to the sub-expressions after conversion.

According to Table 3 we are able to implement the conversion of EXPRESS logical operators, but sometimes the SPARQL statements obtained after the conversion of the expressions on both sides of the logical operators may be relatively complicated, in which case it is necessary to convert the expressions on both sides of the logical operators into the two SPARQL graph patterns respectively. For example, the logical operator *OR* is used in the *WHERE* rule *CorrectTypeAssigned* of the entity *IfcWindow* in Fig. 3. The built-in function *SizeOf* is called in the sub-expression to its left. The SPARQL statement corresponding to this function after conversion is relatively complex, and its conversion process will be described in detail in Section 4.3.2. Therefore, when the SPARQL statements obtained after conversion of expressions on both sides of the logical operators are relatively complex, we propose to consider the EXPRESS logical operator in the *WHERE* rule as the different join between SPARQL graph patterns, which is equivalent to the operation between different sets in set theory. Note that we propose to consider the logical operator *XOR* as a symmetric difference between two sets in set theory and to represent it by other operators *NOT*, *AND* and *OR*. In this case, the conversion patterns of EXPRESS logical operators are shown in Table 4, where *A* and *B* represent sub-expressions on each side of the logical operator in EXPRESS, while *A* and *B* are considered as two graph patterns in SPARQL statements.

$$A \oplus B = (\neg A \wedge B) \vee (A \wedge \neg B) \tag{2}$$

4.2.4. Reference operators

4.2.4.1. Attribute references. The attribute reference is used to refer to a single attribute in an entity instance. The result of the computation of the expression to the left of the attribute reference is an entity instance or a partial complex entity value, and the referenced attribute identifier should be marked after the attribute reference symbol (.) [58]. For example, the EXPRESS specification of IFC entity *IfcGeometricRepresentationContext* is shown in Fig. 9, which defines the context that applies to several shape representations of the products in the project [59]. The *WHERE* rule *North2D* requires that if the entity instance provides a *TrueNorth* direction, it should be a two-dimensional direction [59]. In the entity declaration of *IfcGeometricRepresentationContext*, the data type of the attribute *TrueNorth* is declared as entity data type *IfcDirection*, which means the attribute *TrueNorth* in each instance of the entity *IfcGeometricRepresentationContext* takes the value of one instance of the entity *IfcDirection*. Therefore, the attribute reference in this *WHERE* rule expression is used to return the attribute *DirectionRatios* of the entity *IfcDirection* instance. The value range of this attribute is a list with an upper limit of 3 and a lower limit of 2. According to the conversion pattern in Table 1, the associated RDF graph of the OWL class *IfcGeometricRepresentationContext* is shown in Fig. 10. The attribute *TrueNorth* in the entity *IfcGeometricRepresentationContext* is converted to the object property *trueNorth_IfcGeometricRepresentationContext*. The attribute *DirectionRatios* in the entity *IfcDirection* is converted to the object property *directionRatios_IfcDirection*. The individual *IfcGeometricRepresentationContext_95* is linked to the individual *IfcDirection_93*

Table 3
Conversion pattern of the logical operators.

EXPRESS	SPARQL
A AND B	[c]A&&B
A OR B	[c]A||B
NOT A	[c]!A
A XOR B	[l](!A && B)||(A &&!B)

Table 4
Conversion pattern employed for logical operators in cases where the SPARQL statements are relatively complex.

EXPRESS	SPARQL
A AND B	A. B.
A OR B	{A.} UNION {B.}
NOT A	FILTER NOT EXISTS{A.}

A XOR B	{
	A.
	FILTER NOT EXISTS{B.}
	}
	UNION
	{
	B.
	FILTER NOT EXISTS{A.}
	}

```
ENTITY IfcAxis2Placement3D
SUBTYPE OF (IfcPlacement);
Axis : OPTIONAL IfcDirection;
RefDirection : OPTIONAL IfcDirection;
DERIVE
P : LIST [3:3] OF IfcDirection := IfcBuildAxes(Axis, RefDirection);
WHERE
AxisAndRefDirProvision : NOT ((EXISTS (Axis)) XOR (EXISTS (RefDirection)));
AxisIs3D : (NOT (EXISTS (Axis))) OR (Axis.Dim = 3);
AxisToRefDirPosition : (NOT (EXISTS (Axis))) OR (NOT (EXISTS (RefDirection))) OR
(IfcCrossProduct(Axis,RefDirection).Magnitude > 0.0);
LocationIs3D : SELF\IfcPlacement.Location.Dim = 3;
LocationIsCP : 'IFC4X3.IFCCARTESIANPOINT' IN TYPEOF(SELF\IfcPlacement.Location);
RefDirIs3D : (NOT (EXISTS (RefDirection))) OR (RefDirection.Dim = 3);
END_ENTITY;
```

Fig. 8. EXPRESS specification of the IFC entity *IfcAxis2Placement3D*.

```
ENTITY IfcGeometricRepresentationContext
SUPERTYPE OF (ONEOF(IfcGeometricRepresentationSubContext))
SUBTYPE OF (IfcRepresentationContext);
CoordinateSpaceDimension : IfcDimensionCount;
Precision : OPTIONAL IfcReal;
WorldCoordinateSystem : IfcAxis2Placement;
TrueNorth : OPTIONAL IfcDirection;
INVERSE
HasSubContexts : SET [0:?] OF IfcGeometricRepresentationSubContext FOR ParentContext;
HasCoordinateOperation : SET [0:1] OF IfcCoordinateOperation FOR SourceCRS;
WHERE
North2D : NOT(EXISTS(TrueNorth)) OR (HIINDEX(TrueNorth.DirectionRatios) = 2);
END_ENTITY;
```

Fig. 9. EXPRESS specification of the IFC entity *IfcGeometricRepresentationContext*.

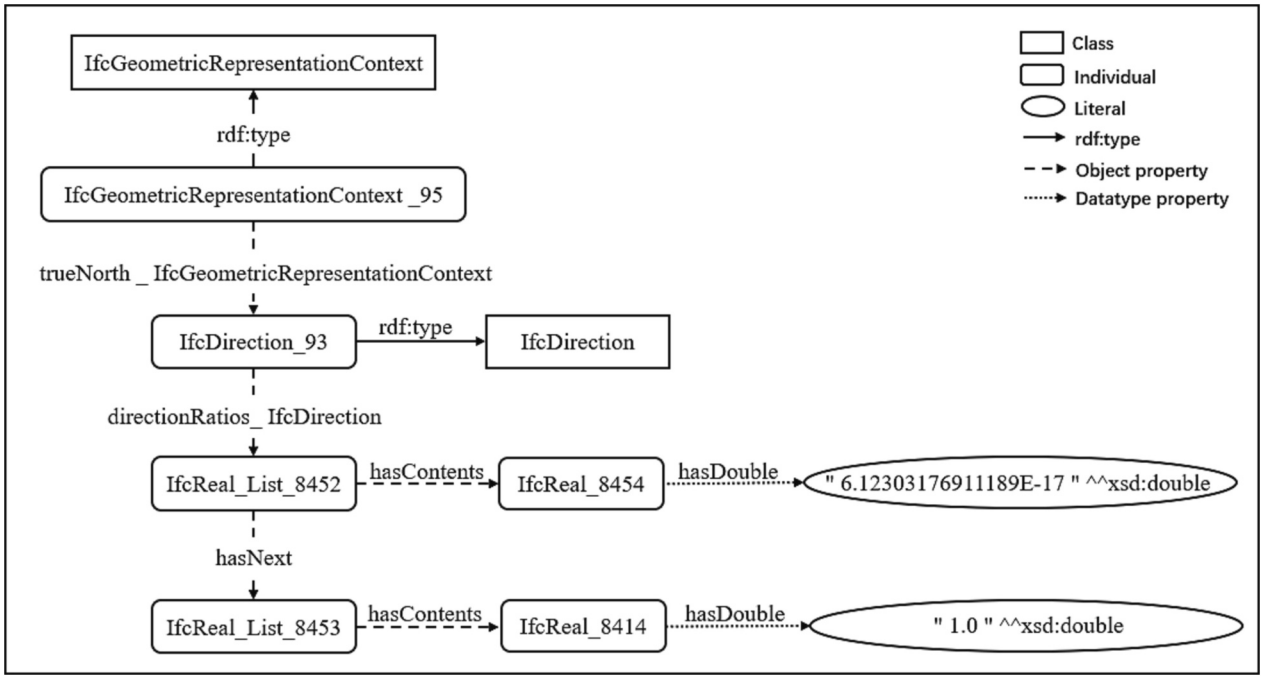


Fig. 10. Associated RDF graph for the OWL class *IfcGeometricRepresentationContext*.

through the object property *trueNorth_IfcGeometricRepresentationContext*, and the individual *IfcDirection_93* is linked to the individual *IfcReal_List_8452* through the object property *directionRatios_IfcDirection*. This individual is the first element in the list, and the second element can be obtained through the object property *express:hasNext*. Therefore, the attribute reference in *North2D* is equivalent to querying the individual *IfcReal_List_8452* linked by the object property *directionRatios_IfcDirection*.

In summary, we propose to achieve similar functionality to the use of attribute reference in the *WHERE* rule by querying the ifcOWL instance graph for the object of specified object property linked with the individual of the target OWL class.

4.2.4.2. Group references. The return value of a group reference is a partial complex entity value. The result of the operation of the expression on the left of the group reference is a complex entity instance, and the entity data type of the quoted partial complex entity value is specified after the group reference symbol (\) [58]. In addition, the group reference can be further qualified by an attribute reference, and the group reference illustrates the scope of the attribute reference [58].

For example, in Fig. 3 the complex entity *IfcWindow* declares the *WHERE* rule *CorrectPredefinedType*, which requires that either the attribute *PredefinedType* is not set, or the inherited attribute *ObjectType* should be provided if *PredefinedType* is set to *USERDEFINED* [59]. The built-in constant *SELF* returns the instance of the complex entity *IfcWindow*. The entity data type *IfcObject* of the referenced partial complex entity value is described after the group reference symbol, where *IfcObject* is the superclass of *IfcWindow*. Finally, the attribute reference further refers to the attribute *ObjectType* in the partial complex entity value. The associated RDF graph of OWL class *IfcWindow* is shown in Fig. 11. The attribute *ObjectType* of *IfcWindow* inherited from its super class *IfcObject* is converted to object property *objectType_IfcObject* in OWL, so we can achieve the reference to the value of the attribute inherited from the abstract class in EXPRESS by querying the object of the object property *objectType_IfcObject* linked with the individual of OWL class *IfcWindow*.

In *WHERE* rules of the IFC schema, a group reference and an attribute reference are often used in combination to further qualify the scope of the attribute reference. Therefore, we propose to implement the reference to the value of an attribute inherited from the abstract class in

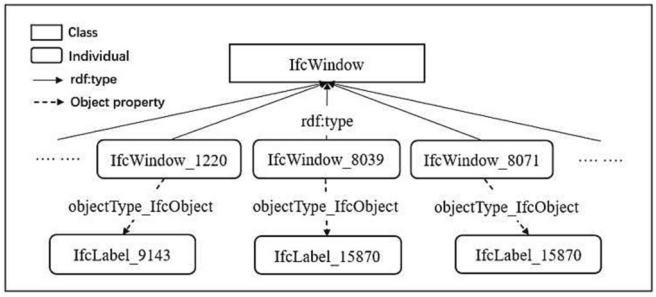


Fig. 11. RDF graph associated with the OWL class *IfcWindow* corresponding to the *WHERE* rule *CorrectPredefinedType*.

EXPRESS by querying the object of the specified object property linked with the individual of OWL class in SPARQL.

4.3. Conversion of built-in functions

4.3.1. TypeOf

In EXPRESS, the function *TypeOf* is used to return the name of the type to which the parameters within the function belong. For example, The *WHERE* rule *CorrectTypeAssigned* in the EXPRESS specification of the entity *IfcWindow* in Fig. 3 requires that either the *IsTypedBy* inverse relationship is not provided or the associated type object must be of type *IfcWindowType* [59]. The sub-expression *('IFC4X3.IFCWINDOWTYPE' IN TYPEOF(SELF\IfcObject.IsTypedBy* [1]*.RelatingType))* is used to verify if the type name*'IFC4X3.IFCWINDOWTYPE'* is included in the data type names returned by the *TypeOf* function, where IFC4X3 represents the name of the IFC schema. The RDF graph associated with the OWL class *IfcWindow* is shown in Fig. 12, where *isTypedBy_IfcObject* and *relatedObjects_IfcRelDefinesByType* are inverse attributes of each other. According to the conversion patterns of built-in constant *SELF* and reference operators in 4.1 and 4.2.4, we can query the ifcOWL instance graph for the value of the attribute *RelatingType* returned by the expression within the function *TypeOf*, i.e. the object of object property *relatingType_IfcRelDefinesByType*, such as *IfcWindowType_1176* and *IfcWindowType_8016*. We can get the type names of these individuals by querying the OWL class they belong to. Therefore, in order to achieve similar functionality to the function *TypeOf* in EXPRESS, we can use the property *rdf:type* in SPARQL to return the OWL class to which the individuals belong in order to get the type name of these individuals.

In summary, using the function *TypeOf* to return the name of the type to which an entity instance belongs in EXPRESS is equivalent to querying the OWL class to which the corresponding individual belongs in the ifcOWL instance graph. Therefore, we recommend using the property *rdf:type* in SPARQL statements to achieve similar functionality of the function *TypeOf*.

4.3.2. HiIndex and SizeOf

There are four aggregation data types defined by EXPRESS, including array data type, list data type, bag data type, and set data type. The function *HiIndex* is used to return the superscript of an array, or the actual number of elements in a bag, set or list [58]. The function *SizeOf* is used to return the number of elements in the aggregation data value, that is, for arrays it returns the number of elements declared, while for bags, sets and lists it returns the actual number of elements [58]. Since the bag data type does not appear in the IFC 4 × 3 schema, and the attributes of the array data type are frequently used for the *DERIVE* attribute, which is beyond the scope of this paper, these two data types were not considered in this study. For set and list data type, both function *HiIndex* and *SizeOf* function are to return the actual number of elements, so the function *HiIndex* and function *SizeOf* can use the same conversion pattern.

The set data type is considered to be an unordered aggregation of similar elements that are different from each other. For example, Fig. 3 shows the EXPRESS specification for the entity *IfcWindow*, which defines a *WHERE* rule called *CorrectTypeAssigned*. The function *SizeOf* in this *WHERE* rule is used to return the actual number of elements within the value of the attribute *IsTypedBy*, which is inherited from the superclass *IfcObject* and takes the value of a set with a lower limit of 0 and an upper limit of 1. As described in Table 1, a common non-functional object property can be assigned to the same instance an unlimited number of times is used to represent the relationship among different connected elements in a set [26]. The associated RDF graph of the OWL class *IfcWindow* is shown in Fig. 12, which contains multiple individuals of this class, such as *IfcWindow_8039*. This individual is linked to the first and only element *IfcRelDefinesByType_12421* in the set by the object property *isTypedBy_IfcObject*. Therefore, we can using SPARQL to query the number of objects linked to individuals of *IfcWindow* through the object property *isTypedBy_IfcObject*, thereby achieving functionality similar to *SizeOf*. In summary, the actual number of elements in the EXPRESS set is calculated by the function *HiIndex* or *SizeOf* in the *WHERE* rule, which is

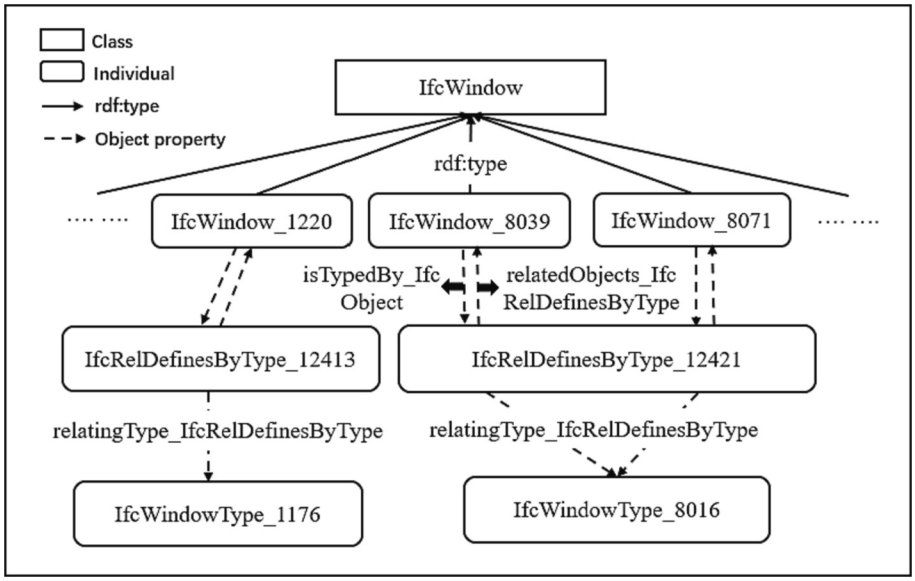


Fig. 12. RDF graph associated with the OWL class *IfcWindow* corresponding to the *WHERE* rule *CorrectTypeAssigned*.

equivalent to querying the number of objects linked to corresponding OWL individuals through a specified object property in ifcOWL.

The list data type is defined as a sequence of similar elements. As an example, the EXPRESS specification of the entity *IfcCartesianPoint* is shown in Fig. 4, the *WHERE* rule *CP2Dor3D* in this entity calls the attribute *Coordinates*. This attribute is a list data type with a lower limit of 1 and an upper limit of 3. The function *HiIndex* returns the actual number of elements in the value of this attribute. According to the conversion pattern in Table 1, each list element is linked with the next element through the object property *express:hasNext*, and linked with its actual content via the object property *express:hasContents* [26]. The associated RDF graph of the OWL class *IfcCartesianPoint* is shown in Fig. 13. The OWL individual *IfcCartesianPoint_10000* forms an RDF triple through the object property *coordinates_lfcCartesianPoint* with the individual *IfcLengthMeasure_List_20324*. The individual *IfcLengthMeasure_List_20324* is the first element in the list, which is linked to the actual value of this element by the object property *express:hasContents*, and the individual is also linked to the second element in the list, *IfcLengthMeasure_List_20325*, by the object property *express:hasNext*, and so on. Therefore, using the function *HiIndex* to calculate the actual number of elements in the value of the attribute *Coordinates* is equivalent to using SPARQL to query the number of all individuals linked directly or indirectly to the first individual *IfcLengthMeasure_List_20324* through the object property *express:hasNext*. In addition, the number of elements should include the first individual itself. In summary, we can query the number of elements linked directly or indirectly to the first list element through the object property *express:hasNext* in ifcOWL to achieve similar functionality to the EXPRESS functions *HiIndex* and *SizeOf*. Property paths in SPARQL provide more concise expressions for basic SPARQL graph patterns, and they also add the ability to match the connectivity of two resources via a path of arbitrary length [61]. Therefore, we can use the property path operator (*) to find all elements that are directly or indirectly linked to the first list element (including the first element itself) via object property *express:hasNext*.

4.3.3. Exists

In EXPRESS, the function *Exists* is usually used to check whether a parameter has a value or is initialized [58]. The function *Exists* returns TRUE if a value exists for the input parameter, FALSE otherwise. For example, there is a *WHERE* rule *CorrectPredefinedType* in the EXPRESS specification of the entity *IfcWindow* in Fig. 3, where the function *Exists* is called to check whether the attribute *PredefinedType* has been assigned an appropriate value in instances of the entity. There is also a function named *Exists* in SPARQL that acts as a filter operator and takes the graph pattern as an argument [61]. If the graph pattern matches, it returns TRUE, otherwise it returns FALSE. Therefore, we propose to use the SPARQL function *Exists* in ifcOWL to achieve similar functionality to the EXPRESS function *Exists* in the *WHERE* rule.

5. Conversion of *WHERE* rules to SHACL constraints

Based on the conversion patterns of built-in constant, operators and built-in functions commonly used in EXPRESS described in Section 4, this section elaborates on the process of converting *WHERE* rules in entities and types into SPARQL-based SHACL constraints. The conversion process can be divided into the following three steps:

1. According to the conversion patterns for built-in constant, operators, and built-in functions, the *WHERE* rules in types or entities are converted into SPARQL statements.
2. For each type or entity with *WHERE* rules, one or more SHACL shapes are created for the corresponding OWL classes in the ifcOWL instance graph, and the SPARQL statements from Step 1 are added to these shapes.
3. Link the shape defined in Step 2 to the target OWL class to which the *WHERE* rule belongs.

Furthermore, it is crucial to note that after converting *WHERE* rules into SPARQL statements, our aim is to identify individuals in the ifcOWL instance graph that do not conform to these *WHERE* rules. Therefore, we might consider adding a *FILTER NOT EXISTS* expression in the SPARQL statements. The purpose of this is to exclude those individuals that already meet the *WHERE* rules, thereby ensuring that the SHACL validation results display only those individuals that fail to satisfy the *WHERE* rules.

5.1. Conversion of WHERE rules in IFC types

In the IFC schema, some type declarations use the *WHERE* rule as a further constraint on the type. We divide the IFC types containing

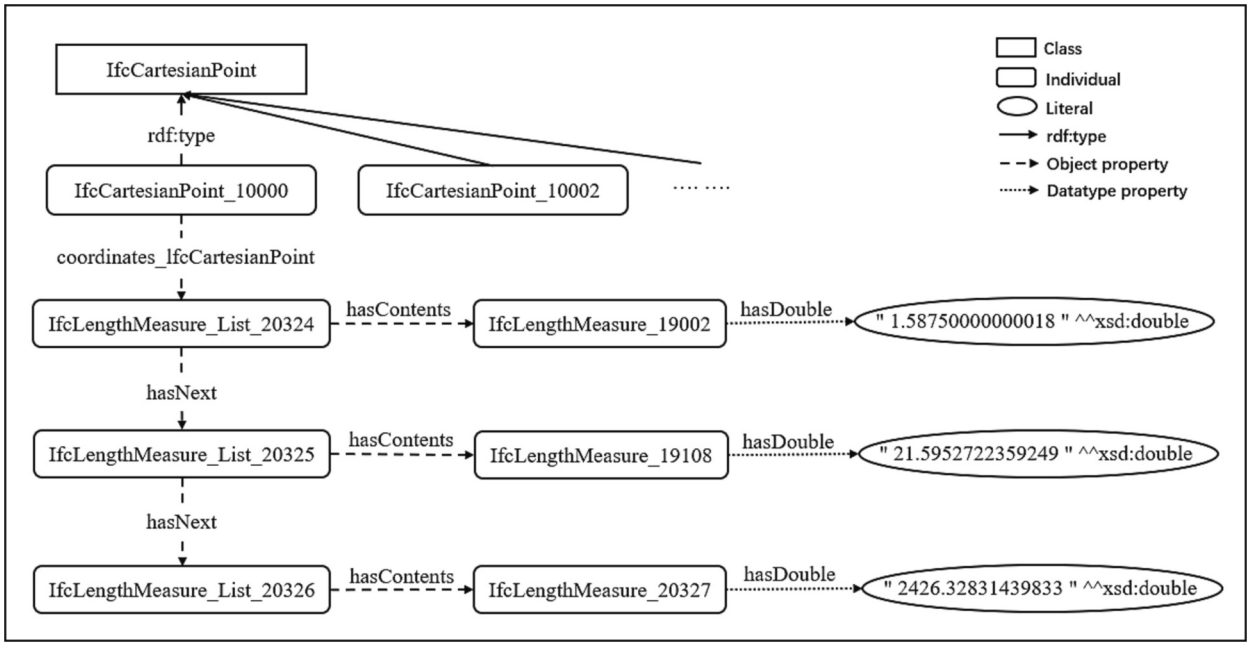


Fig. 13. Associated RDF graph of OWL class *IfcCartesianPoint*.

WHERE rules into two categories, where one kind of *WHERE* rules specifies the numerical range of the type value, and the other kind specifies the optional string of the type value. In this section, we present the conversion pattern of *WHERE* rules to SPARQL-based SHACL constraints using examples of both kinds of type declarations, which cover the conversion methods of all *WHERE* rules in IFC types.

For the first category of IFC types, for example, the EXPRESS specification of the IFC type *IfcPositiveLengthMeasure* we have given in Fig. 2, where the *WHERE* rule *WR1* constrains that the positive measure should be greater than zero. Based on the RDF graph associated with the OWL class *ifc:IfcPositiveLengthMeasure* in Fig. 5, we give the definition of the SPARQL-based SHACL shape corresponding to this *WHERE* rule in Fig. 14. This *WHERE* rule uses built-in constant *SELF* and value comparison operator, so based on the conversion patterns of *SELF* in Section 4.1 and the value comparison operator in Section 4.2.2.1, we query all individuals of the OWL class *ifc:IfcPositiveLengthMeasure* and query objects linked to individuals through the datatype property *express:hasDouble* in ifcOWL instance graph. We use *NOT EXISTS* in the *FILTER* expression to to return solutions that do not meet the constraint of "positive measure should be greater than zero" specified in the *WHERE* rule. After obtaining the SPARQL-based constraint, we add the constraint to the established node shape *ifc:WR1_IfcPositiveLengthMeasure* via the SHACL property *sh:sparql* and link the shape to the target OWL class *ifc:IfcPositiveLengthMeasure* via the SHACL property *sh:targetClass*.

For the second category of IFC types, for example, the type *IfcTextAlignment* defines how the text is aligned within the element. Fig. 7 shows its EXPRESS specification, in which the *WHERE* rule *WR1* states that the allowable text alignment values are "left", "right", "center" and "justify". Fig. 15 shows the definition of the SPARQL-based SHACL shape corresponding to this *WHERE* rule. The membership operator is converted to the function *IN* in SPARQL according to the conversion pattern introduced in Section 4.2.2.2 to test whether the value of the individual can be found in the strings "left", "right", "center" and "justify". We use *NOT EXISTS* in the *FILTER* expression to return solutions that do not satisfy the constraint specified in this *WHERE* rule. After getting the SPARQL constraint, we add this constraint to the created shape *ifc:WR1_IfcTextAlignment* via SHACL property *sh:sparql* and specify the target class of this shape as *ifc:IfcTextAlignment* via the SHACL property *sh:targetClass*.

```
ifc:WR1_IfcPositiveLengthMeasure
  rdf:type sh:NodeShape ;
  sh:sparql
  [
     sh:message "A positive measure shall be greater than zero." ;
     sh:prefixes <http://linkedbuildingdata.net/ifc/resources/20230316_152014> ;
     sh:select
     """
         SELECT $this ?value
         WHERE
         {
            $this rdf:type ifc:IfcPositiveLengthMeasure .
            OPTIONAL {$this express:hasDouble ?value} .
            FILTER NOT EXISTS
            {
               FILTER( ?value > 0)
            }
         }
     """ ;
  ] ;
  sh:targetClass ifc:IfcPositiveLengthMeasure ;
.
```

Fig. 14. SPARQL-based SHACL constraint for the *WHERE* rule *WR1* in *IfcPositiveLengthMeasure*.

```
ifc:WR1_IfcTextAlignment
  rdf:type sh:NodeShape ;
  sh:sparql
  [
     sh:message "The allowable values for text alignment can only be one of 'left', 'right',
'center' or 'justify'" ;
     sh:prefixes <http://linkedbuildingdata.net/ifc/resources/20230316_152014> ;
     sh:select
     """
        SELECT $this ?value
        WHERE
        {
           $this rdf:type  ifc:IfcTextAlignment .
           OPTIONAL {$this express:hasString ?value} .
           FILTER NOT  EXISTS
           {
              FILTER( ?value IN ('left' ,'right' , 'center', 'justify' ))
           }
        }
     """ ;
  ] ;
  sh:targetClass ifc:IfcTextAlignment ;
.
```

Fig. 15. SPARQL-based SHACL constraint for the *WHERE* rule *WR1* in *IfcTextAlignment*.

5.2. Conversion of WHERE rules in IFC entities

In IFC schema, there are a large number of entity declarations that use *WHERE* rules as further constraints. Similar to the conversion process for *WHERE* rules in type declarations, we need to convert the constraint defined by the *WHERE* rule in entity declaration to a SPARQL-based constraint, add the constraint to a SHACL shape and specify the target class of the shape. However, unlike type declarations, there are some superclass entities in the IFC schema with *ABSTRACT SUPERTYPE* declarations. These abstract superclass entities serve as the abstract base classes for their instantiable subclass entities in the IFC schema. Typically, IFC STEP files primarily include directly instantiable subclass entities such as *IfcWall* and *IfcWindow*, among others. Therefore, they are usually absent in IFC STEP files. However, in the IFC schema, subclass entities inherit the attributes and constraints defined by their superclass entities. To achieve comprehensive data validation, we chose to convert the *WHERE* rules defined by these superclass entities into SHACL shapes and associate them with the corresponding OWL classes of all their instantiable subclass entities. In this section, we will list some of the entities used as examples in Section 4 to show how to convert *WHERE* rules within entities to SPARQL-based SHACL constraints.

Take the entity *IfcWindow* as the first example, it represents the window in the building element. Fig. 3 shows the definition of two *WHERE* rules *CorrectPredefinedType* and *CorrectTypeAssigned* in the EXPRESS specification of *IfcWindow*. In the core data schema, shared element data schema and domain specific data schema of the IFC data schema architecture, there are a large number of entities that define these two types of *WHERE* rules, so the same conversion methods can be used.

The semantic interpretation of the *WHERE* rule *CorrectPredefinedType* for the IFC entity *IfcWindow* is that either the attribute *PredefinedType* is not set, or the inherited attribute *ObjectType* should be provided, if the *PredefinedType* is set to *USERDEFINED* [59]. According to the RDF graph associated with the OWL class *ifc:IfcWindow* in Fig. 11, the definition of the SPARQL-based SHACL shape corresponding to this *WHERE* rule is shown in Fig. 16. For the logical operators *NOT*, *OR* and *AND*, since the SPARQL statements obtained by sub-expressions on both sides of these operators in this *WHERE* rule are relatively simple after conversion, we can convert these operators according to Table 3. For the value comparison operators "<>"and "="in the *WHERE* rule, we convert them to the SPARQL operators "! = "and "="according to Table 2. For the built-in function *EXISTS*, we convert it to the SPARQL function *EXISTS* according to the conversion pattern in Section 4.3.3, while the built-in constant *SELF* and the reference operator are

```
ifc:CorrectPredefinedType_IfcWindow
 rdf:type sh:NodeShape ;
 sh:sparql [
    sh:message "Either the PredefinedType attribute is unset, or the inherited attribute ObjectType shall be provided, if the
PredefinedType is set to USERDEFINED." ;
    sh:prefixes <http://linkedbuildingdata.net/ifc/resources/20230316_152014> ;
    sh:select """
    SELECT $this ?object_1 ?object_2
    WHERE
    {
       $this rdf:type  ifc:IfcWindow .
       OPTIONAL { $this   ifc:predefinedType_IfcWindow ?object_1} .
       OPTIONAL { $this ifc:objectType_IfcObject  ?object_2} .
       FILTER NOT EXISTS
       {
         FILTER ( !EXISTS{$this  ifc:predefinedType_IfcWindow  ?object_1}  || (?object_1  !=  ifc:USERDEFINED)  ||
( (?object_1  =  ifc:USERDEFINED)  &&  EXISTS{$this  ifc:objectType_IfcObject   ?object_2}) )
       }
    }""" ;
 ] ;
 sh:targetClass ifc:IfcWindow ;
.
```

Fig. 16. SPARQL-based SHACL constraint for the *WHERE* rule *CorrectPredefinedType* in *IfcWindow*.

converted according to Sections 4.1 and 4.2.4. We use *NOT EXISTS* in the *FILTER* expression to return solutions that do not satisfy the constraint defined by the *WHERE* rule *CorrectPredefinedType*. After establishing a constraint based on SPARQL that can validate the *WHERE* rule, we add the constraint to the established node shape *ifc:CorrectPredefinedType_IfcWindow* via the SHACL property *sh:sparql*, and specify the target class of the shape as *ifc:IfcWindow* via the SHACL property *sh:targetClass*.

For the *WHERE* rule *CorrectTypeAssigned* in the IFC entity *IfcWindow*, the semantic interpretation of this constraint is that either the *IsTypedBy* inverse relationship is not provided, or the associated type object must be of type *IfcWindowType* [59]. According to the RDF graph associated with the OWL class *ifc:IfcWindow* in Fig. 12, the definition of the SPARQL-based SHACL shape corresponding to this *WHERE* rule is shown in Fig. 17. Because of the relative complexity of the SPARQL statement

```
ifc:CorrectTypeAssigned_IfcWindow
 rdf:type sh:NodeShape ;
 sh:sparql [
    sh:message "Either the IsTypedBy inverse relationship is not provided, or the associated type object must be of type IfcWindowType." ;
    sh:prefixes <http://linkedbuildingdata.net/ifc/resources/20230316_152014> ;
    sh:select """
      SELECT $this
      WHERE
      {
        $this rdf:type ifc:IfcWindow .
        FILTER NOT EXISTS
        {
          {
            SELECT $this
            WHERE
            {
              $this rdf:type ifc:IfcWindow .
              OPTIONAL {$this ifc:isTypedBy_IfcObject  ?object_1} .
            }
           GROUP BY $this
           HAVING(COUNT(?object_1)=0)
          }
          UNION
          {
            $this ifc:isTypedBy_IfcObject  ?object_1.
            ?object_1 ifc:relatingType_IfcRelDefinesByType ?object_2 .
            ?object_2 rdf:type ?object_3.
            FILTER( ifc:IfcWindowType IN (?object_3))
          }
        }
    }""" ;
 ] ;
 sh:targetClass ifc:IfcWindow ;
.
```

Fig. 17. SPARQL-based SHACL constraint for the *WHERE* rule *CorrectTypeAssigned* in *IfcWindow*.

after conversion of the sub-expressions on both sides of the logical operator *OR*, this logical operator can be converted to the keyword *UNION* according to Table 4, and the sub-expressions on both sides are converted to two graph patterns. It is worth noting that the *WHERE* rule expression calls the built-in EXPRESS functions *Sizeof* and *Typeof*. The function *Sizeof* is used to calculate the number of elements in the value of the attribute *IsTypedBy*, the attribute is a set data type with a lower limit of 0 and an upper limit of 1, so there is at most one element in the attribute value. The *OPTIONAL* keyword is used to ensure that when the object property *ifc:isTypedBy_IfcObject* does not have a linked object, i.e. when the number of elements within the value of the attribute *IsTypedBy* is 0, the solution is not eliminated even if no binding is created in SPARQL. In addition, the *GROUP BY* clause is used to group solutions according to different individuals of OWL class *ifc:IfcWindow*, and then these groups are filtered by *HAVING* expression. When the object of object property *ifc:isTypedBy_IfcObject* does not exist, the group corresponding to the individual of *ifc:IfcWindow* will be deleted. The SPARQL set function *COUNT* in the *HAVING* clause is used to count the number of solutions in each group, that is, the number of elements in the value of each attribute *isTypedBy*. For the built-in function *Typeof*, we can use the RDF property *rdf:type* according to the conversion pattern in Section 4.3.1 to achieve similar functionality as this function. We use *NOT EXISTS* in the *FILTER* expression to return solutions that do not satisfy the constraint defined by the *WHERE* rule *CorrectTypeAssigned*. After establishing a constraint based on SPARQL that can validate the *WHERE* rule, we add the constraint to the established node shape *ifc:CorrectTypeAssigned_IfcWindow* via the SHACL property *sh:sparql*, and specify the target class of the shape as *ifc:IfcWindow* via the SHACL property *sh:targetClass*.

As a further example, the EXPRESS specification of the entity *IfcCartesianPoint* is shown in Fig. 4, which defines a *WHERE* rule called *CP2Dor3D* that invokes the attribute *Coordinates* of the list data type and requires that there be at least two elements in the value of the attribute. It is worth noting that this *WHERE* rule calls the built-in function *HiIndex*, which takes the attribute *Coordinates* as an argument and returns the number of elements in the attribute value. Based on the conversion pattern of the function *HiIndex* in Section 4.3.2 and the associated RDF graph of the OWL class *ifc:IfcCartesianPoint* in Fig. 13, we give the SPARQL-based SHACL constraint for this *WHERE* rule, as shown in Fig. 18. We query the first element in the value of the attribute *Coordinates* by the object property *ifc:coordinates_IfcCartesianPoint* in SPARQL, and then use the property path operator "*" to query all elements linked with the first element by the object property *express:hasNext* (including the first element itself). In addition, the *GROUP BY* clause is used to group solutions according to different individuals of the OWL class *ifc:IfcCartesianPoint*, and the *HAVING* clause is used to filter the grouped set of solutions, all groups with less than 2 elements will be deleted. We use *NOT EXISTS* in the *FILTER* expression to return solutions that do not satisfy the constraint defined by the *WHERE* rule *CP2Dor3D*. After establishing a constraint based on SPARQL that can validate the *WHERE* rule, we add the constraint to the established node shape *ifc:CP2Dor3D_IfcCartesianPoint* via the SHACL property *sh:sparql*, and specify the target class of the shape as *ifc:IfcCartesianPoint* via the SHACL property *sh:targetClass*.

```
ifc:CP2Dor3D_IfcCartesianPoint
 rdf:type sh:NodeShape ;
 sh:sparql [
    sh:message "Only two or three dimensional points are in scope." ;
    sh:prefixes <http://linkedbuildingdata.net/ifc/resources/20230316_152014> ;
    sh:select """SELECT $this
    WHERE
    {
            $this rdf:type ifc:IfcCartesianPoint .
        FILTER NOT EXISTS
        {
            SELECT  $this
            WHERE
            {
                $this rdf:type ifc:IfcCartesianPoint .
                $this ifc:coordinates_IfcCartesianPoint ?sup .
                OPTIONAL{ ?sup  list:hasNext* ?sub}.
            }
            GROUP BY $this
            HAVING (count($this) >=2)
        }
    }""" ;
  ] ;
 sh:targetClass ifc:IfcCartesianPoint ;
.
```

Fig. 18. SPARQL-based SHACL constraint for the *WHERE* rule *CP2Dor3D* in *IfcCartesianPoint*.

As a final example, the EXPRESS specification of the entity *IfcAxis2Placement3D* is given in Fig. 8, which defines a *WHERE* rule named *AxisAndRefDirProvision*. This *WHERE* rule requires that either the attribute *Axis* and the attribute *RefDirection* are not given or both are given [59]. The *WHERE* rule calls the built-in function *Exists*, the logical operators *NOT* and *XOR*. The definition of the SPARQL-based SHACL shape corresponding to this *WHERE* rule is shown in Fig. 19. For the built-in function *Exists* we can convert it according to Section 4.3.3. It is worth noting that since the sub-expressions connected by the logical operators *NOT* and *XOR* obtain relatively simple SPARQL statements after conversion, we can convert them according to the conversion pattern summarized in Table 3. We use *NOT EXISTS* in the *FILTER* expression to return solutions that do not satisfy the constraint defined by the *WHERE* rule *AxisAndRefDirProvision*. We then add the SPARQL-based constraint to the node shape *ifc:AxisAndRefDirProvision_IfcAxis2Placement3D* and specifies the target class of the shape as *ifc:IfcAxis2Placement3D* via the SHACL property *sh:targetClass*.

6. SHACL validation for ifcOWL

To automate the processing of a large number of *WHERE* rules, we have developed a Python program that can parse the *WHERE* rules defined in EXPRESS specifications and convert them into SHACL shapes. Additionally, the program can validate ifcOWL instance graphs against the *WHERE* rules. The program framework is shown in Fig. 20. We use web scraping techniques to extract all entities and types that contain *WHERE* rules from the IFC documentation. Then, we apply regular expressions to parse the EXPRESS specification for each entity and type, breaking down the different parts of the *WHERE* rules and extracting relevant information. Based on the conversion patterns of *WHERE* rules described in Section 4, we have established mapping rules to convert built-in constant, operators, and built-in functions in *WHERE* rules into SPARQL statements. Using these mapping rules, we can convert *WHERE* rules into SPARQL queries. To provide an end-to-end validation solution, our program calls two Python libraries during the data validation stage: PySHACL and RDFLib. PySHACL utilizes automatically generated shapes graph to validate the user-imported ifcOWL instance graph and generates corresponding validation results. Meanwhile, RDFLib helps us parse and manipulate the ifcOWL instance graph, and assists in converting rules into SPARQL statements.

For users, all they need to do is select and import the ifcOWL instance graph in the user interface, and a validation report will be generated. We used an ifcOWL instance graph for the validation demo, which contains a total of 27,712 OWL individuals. The partial structure of the file in the ontology editor TopBraid Composer is shown in Fig. 21, and the corresponding IFC file is shown in Fig. 22. After importing the file into the program, the validation results are displayed in the text box on the user interface, as shown in Fig. 23. The validation report includes the number of OWL individuals that violate the *WHERE* rules and detailed error information for each violated rule of the OWL individuals. Users can export the validation report in text format to the local file system and make further modifications and improvements to the ifcOWL instance

```
ifc:AxisAndRefDirProvision_IfcAxis2Placement3D
 rdf:type sh:NodeShape ;
 sh:sparql [
    sh:message "Either both, Axis and RefDirection are not given and therefore defaulted, or both shall be given." ;
    sh:prefixes <http://linkedbuildingdata.net/ifc/resources/20230316_152014> ;
    sh:select """SELECT $this
    WHERE
    {
       $this rdf:type ifc:IfcAxis2Placement3D
       OPTIONAL {$this  ifc:axis_IfcAxis2Placement3D ?object_1}
       OPTIONAL{$this  ifc:refDirection_IfcAxis2Placement3D ?object_2}
       FILTER NOT EXISTS
       {
          FILTER(! ((!EXISTS{$this  ifc:axis_IfcAxis2Placement3D  ?object_1  &&  EXISTS{$this
ifc:refDirection_IfcAxis2Placement3D ?object_2} ) || ( EXISTS{$this  ifc:axis_IfcAxis2Placement3D  ?object_1}  &&
!EXISTS{$this  ifc:refDirection_IfcAxis2Placement3D ?object_2})))
       }
    }""" ;
  ] ;
 sh:targetClass ifc:IfcAxis2Placement3D ;
.
```

Fig. 19. SPARQL-based SHACL constraint for the *WHERE* rule *AxisAndRefDirProvision* in *IfcAxis2Placement3D*.

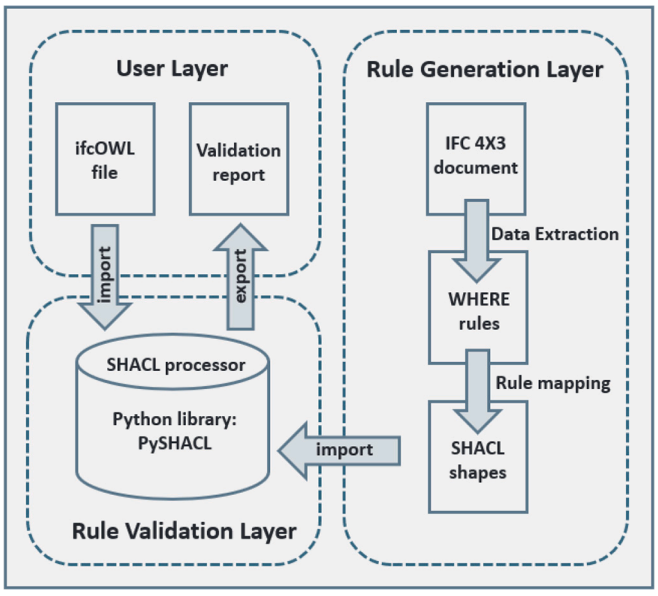


Fig. 20. Program framework for ifcOWL validation.

graph based on the error information in the report.

In the validation report of the ifcOWL instance graph, there is a violation message related to the individual *inst:IfcWindow_1220*. The violation message indicates that this individual does not comply with the constraint defined by the SHACL shape *ifc:CorrectTypeAssigned_IfcWindow*, which means it violates the *WHERE* rule *CorrectTypeAssigned* defined within the IFC entity *IfcWindow*. To verify the accuracy of this error message, we opened the resource forms of the individuals *inst:IfcWindow_1220* and *inst:IfcRelDefinesByType_12399* in TopBraid Composer, as shown in Fig. 24. In the resource form of *inst:IfcWindow_1220*, the object of the object property *ifc:isTypedBy_IfcObject* is *inst:IfcRelDefinesByType_12399*. In the resource form of *inst:IfcRelDefinesByType_12399*, the object of the object property *ifc:relatingType_IfcRelDefinesByType* is *inst:IfcWallType_229*, which is an individual of the OWL class *ifc:IfcWallType*. However, the *WHERE* rule *CorrectTypeAssigned* of *IfcWindow* requires that the associated type object of *IfcWindow* must be *IfcWindowType*. Therefore, the individual *inst:IfcWindow_1220* does not satisfy the constraint defined by the *WHERE* rule *CorrectTypeAssigned*.

The experimental results demonstrate that the SHACL constraints generated by our program accurately match the *WHERE* rule definitions in the original IFC schema, and it can perform constraint validation for the entire ifcOWL instance graph based on *WHERE* rules in a short period of time. This program provides a novel, efficient, and reliable solution for automating the validation of ifcOWL instance graphs.

7. Discussion

After conducting a comprehensive review of the EXPRESS specifications for all types and entities in the IFC4X3 documentation, we found that within the four conceptual layers of the IFC schema, a total of 367 entities and 20 types have defined *WHERE* rules. In total, there are 591 *WHERE* rules included. During the rule generation phase, the generated SHACL constraints include all *WHERE* rules from the IFC types, as well as up to 73.2% of *WHERE* rules from IFC entities. All of the aforementioned SHACL shapes are saved as shapes graph within the program, which can be utilized for comprehensive checks on ifcOWL instance graphs imported into the program. When users import an ifcOWL instance graph into the program, the SHACL processor will validate all individuals of the target OWL classes in the file according to the shape graph. The validation results, along with relevant information, are reported to the users through the user interface.

To ensure the performance and accuracy of the program, we conducted a series of tests involving multiple ifcOWL instance graphs. These test files simulate real-world scenarios, encompassing different types and scales of buildings, ranging from small residential homes to large commercial projects. For example, the test files include a complex ifcOWL instance graph with a sophisticated building structure, containing a significant amount of data related to walls, windows, and other building components. In our testing, the shape-based validation tool accurately identified data that did not comply with *WHERE* rules. Additionally, when handling complex files containing thousands of entities, this tool typically completed a comprehensive validation process in just a few minutes on average. Throughout the validation process, we meticulously recorded execution times, the number of issues discovered, and the types of errors for each test case. The tests demonstrate that our program exhibits a high level of accuracy and efficiency in identifying data non-compliance with *WHERE* rules. In summary, this program serves as an additional validation layer that can be used at different stages of project development. It provides users with a reliable tool to inspect and rectify potential data quality issues, thereby enhancing the reliability of data integration and interoperability between the BIM domain and other domains.

Compared to the approach taken by Zhao et al. [48] using complex SWRL rules to convert *WHERE* rules, our method demonstrates

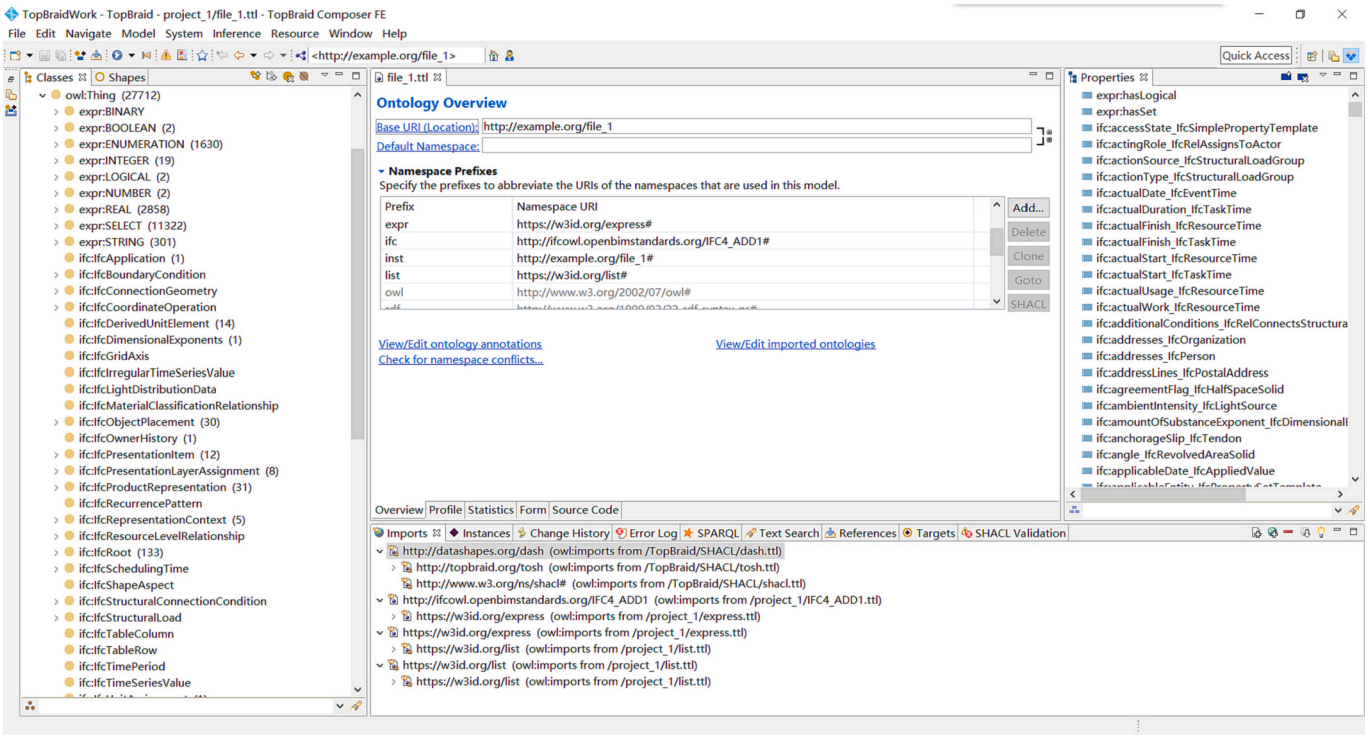


Fig. 21. Partial structure of the sample ifcOWL instance graph in the ontology editor TopBraid Composer.

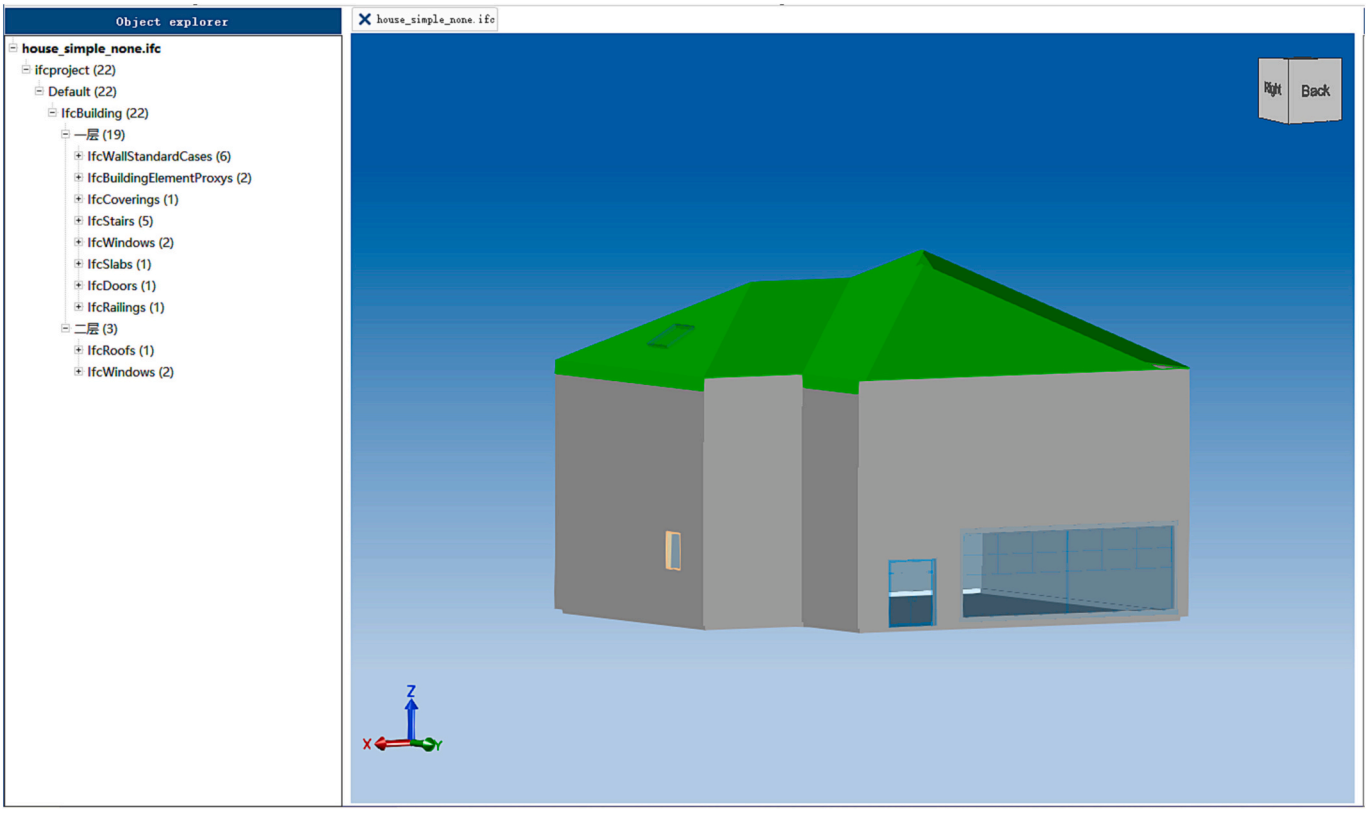


Fig. 22. Corresponding IFC file for the sample ifcOWL instance graph in Open IFC Viwer.

superiority in several aspects. Firstly, the syntax and 77semantics of SWRL are more complex for non-semantic web developers, which not only increases development time but also adds complexity to maintenance tasks. Secondly, considering that ifcOWL instance graphs typically contain a large number of triples, the use of SWRL rules for reasoning can impose a significant burden on the inference engine, thereby compromising performance. In contrast, our program is designed to simplify the user experience, where users can simply import

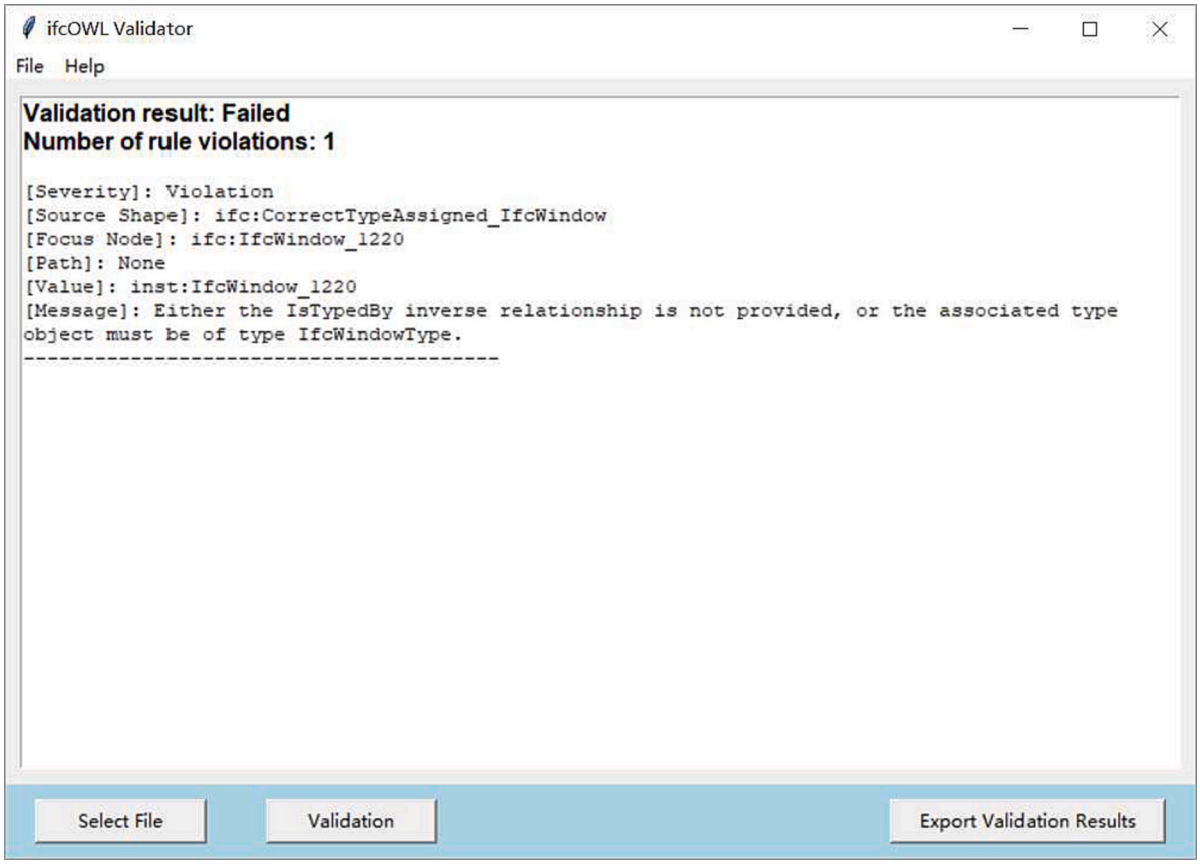


Fig. 23. Validation results of the sample ifcOWL instance graph.

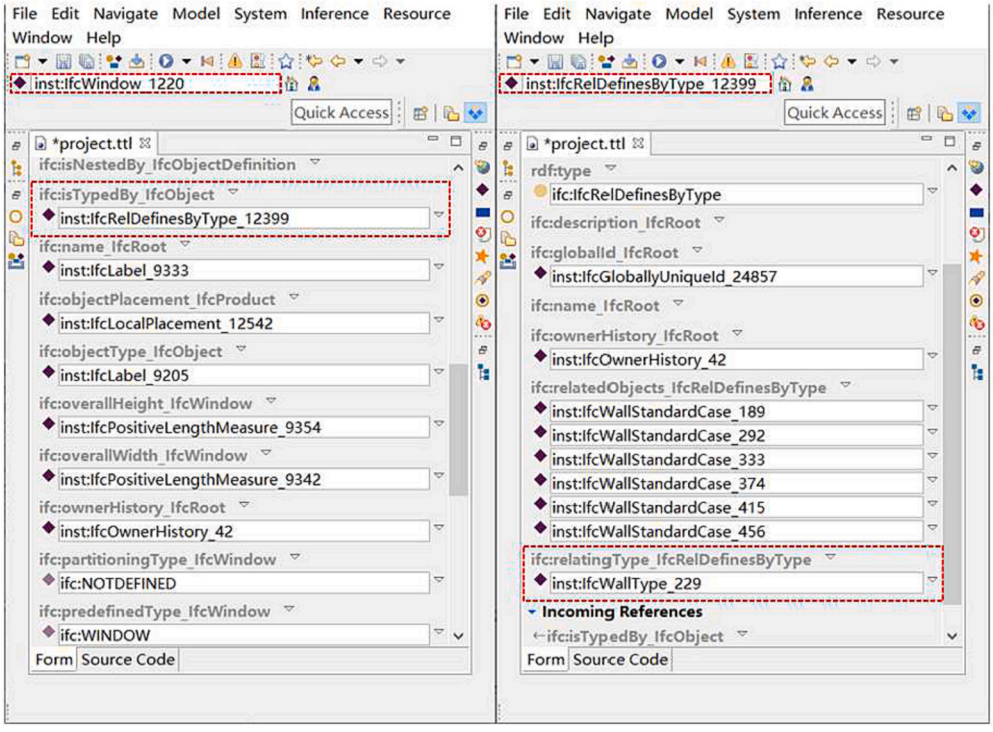


Fig. 24. Resource Forms of *inst:IfcWindow_1220* and *inst:IfcRelDefinesByType_12399*.

the file for validation without the need for additional coding or adjustments. Particularly, given that SHACL is specifically designed for validating RDF datasets, it provides a more direct and efficient approach to defining and validating constraints on data models, thus surpassing other methods in terms of performance. Compared to the strategy adopted by Terkaj et al. [44] of converting certain *WHERE* rules into OWL using class expressions, our method has significantly expanded the scope of validation. It is no longer limited to only converting rules related to the entities *IfcRelDefinesByType* and *IfcPreDefinedPropertySet*, and can effectively handle more complex rules. Moreover, compared to their approach of partially modifying the conversion rules defined by Pauwels et al. [26], our program design does not require any modifications to the original conversion standards. This means that we can perform accurate validation without altering the original structure of ifcOWL. Overall, our method offers greater feasibility and flexibility compared to previous approaches, especially when dealing with the validation of *WHERE* rules in practical application scenarios.

There are still some unresolved issues in our program that need further discussion. Currently, 26.8% of *WHERE* rules have not been converted in our program. First of all, some of *WHERE* rules involve *DERIVE* attributes and IFC functions. *DERIVE* attributes require calculations based on expressions involving other attributes, while IFC functions are algorithms that operate on parameters. Both their definitions are complex and their conversion methods are beyond the scope of this research, so they have been ignored in our study. Additionally, some built-in EXPRESS functions (*QUERY* and *BLENGTH*) and EXPRESS instance comparison operators (:<>: and:=:) have also not been covered in our conversion patterns. The conversion of these built-in functions and operators involves more complex syntax and semantics, and may require additional handling and parsing. Due to time and resource constraints, we did not address the conversion of these structures in our current research. Therefore, these unaddressed conversion methods will be considered as directions for future research to enhance the functionality and coverage of our validation program.

8. Conclusions and future work

To ensure precise and efficient planning, decision-making, and safe operational management, the need for interoperability and integration between BIM and other domains is increasingly significant. Against this backdrop, ontological technology has emerged as an ideal choice for achieving information flow integration and interoperability. Through ontological technology, cross-domain models with shared semantics can be constructed, allowing data from different systems to interact and integrate at the semantic level. In recent years, ifcOWL has become a research focus and has been widely applied to address the issues of cross-system integration and interoperability in the BIM field. However, despite its broad application prospects, ifcOWL currently does not fully support the *WHERE* rules in the IFC schema. The *WHERE* rules in IFC schema are key to defining data integrity and logical consistency. If these *WHERE* rules are neglected or misinterpreted when applying ifcOWL, errors and inconsistencies may be introduced during data conversion. Therefore, to guarantee data integrity and accuracy, it is imperative to further refine and optimize ifcOWL during the integration and interaction of BIM with other domains, ensuring it fully adheres to and supports all the rules within the IFC schema.

Recognizing the critical role of *WHERE* rules in maintaining data integrity and logical consistency, this paper introduces an innovative approach that converts these rules into SHACL constraints based on the SPARQL query language. This ensures that the rules are more completely and accurately represented within ifcOWL. By converting *WHERE* rules into shape graphs and associating them with specific target classes in ifcOWL, an additional layer of validation is introduced. This layer is designed to meticulously validate that the data in ifcOWL instance graphs adheres strictly to the rules and standards defined by the IFC schema. Through this stringent validation process, the likelihood of errors during data conversion is significantly reduced, ensuring the accuracy and reliability of information exchange across various domains.

Based on the conversion schema we proposed, we have successfully developed an automated validation tool that parses *WHERE* rules from IFC schema and converts them into SHACL shapes associated with target OWL classes. This tool offers a user-friendly interface, allowing users to simply import ifcOWL instance graphs to effortlessly obtain detailed validation results. AEC professionals can rely on our validation reports to effectively identify, manage, and correct potential errors that arise during data conversion, project iteration, and project expansion processes. With pre-validated data, the exchange of information and decision-making processes across multiple disciplines can also be accelerated, as stakeholders can trust the data more, assured that there are no underlying inconsistencies or mistakes.

At the current stage of research, we are focusing on implementing and validating the simpler and more common *WHERE* rules within IFC schema, which are crucial for ensuring data integrity and logical consistency. Therefore, our research has not yet encompassed the *WHERE* rules that involve complex *DERIVE* attributes and IFC functions. This limitation suggests that there is still a significant amount of work to be done in the domain of rule conversion and validation for IFC schema. Our work lays a solid foundation for future research, which can be further deepened and expanded upon. We plan to further optimize the functionality of our conversion tools in future work, striving to incorporate those *WHERE* rules not yet converted into our validation framework, thereby providing users with a more comprehensive and detailed data validation tool. Within the AEC sector, SHACL, as an emerging technology, is demonstrating its unique potential and advantages. A promising direction for research is the integration of SHACL with knowledge graphs and deep learning technologies to explore new paradigms of data validation. For instance, by utilizing natural language processing technology, SHACL could enable the automated validation of building models against building codes to ensure model accuracy. Such research could effectively reduce the costs of manual validation and improve the efficiency and accuracy of building code compliance checks. In summary, SHACL is poised to offer more efficient and intelligent data validation solutions for the AEC industry in the future, and further advancing integrated planning, decision support, and efficient management within the field.

CRediT authorship contribution statement

Chaoyue Wang: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Liang Zhang:** Writing – review & editing, Writing – original draft, Supervision, Resources, Project administration, Methodology, Funding acquisition. **Wei Yan:** Writing – review & editing, Writing – original draft, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This study was supported by the National Natural Science Foundation of China (No.62002207, No.62072290, No.62073201) and Shandong Provincial Natural Science Foundation(No. ZR2022YQ05, No.

ZR2020MA102).

References

[1] Sara Shirowzhan, Samad M.E. Sepasgozar, David J. Edwards, Heng Li, Chen Wang, BIM compatibility and its differentiation with interoperability challenges as an innovation factor, Autom. Constr. 112 (2020), https://doi.org/10.1016/j.autcon.2020.103086, 103086.

[2] Hafiz Muhammad Faisal Shehzad, Roliana Binti Ibrahim, Ahmad Fadhil Yusof, Khairul Anwar Mohamed Khaidzir, Muhammad Iqbal, Saad Razzaq, The role of interoperability dimensions in building information modelling, Comput. Ind. 129 (2021) 103444, https://doi.org/10.1016/j.compind.2021.103444.

[3] Yusuf Arayici, Terrence Fernando, Victor Munoz, May Bassanino, Interoperability specification development for integrated BIM use in performance based design, Autom. Constr. 85 (2018) 167–181, https://doi.org/10.1016/j.autcon.2017.10.018.

[4] Xiao Li, Wu Peng, Geoffrey Qiping Shen, Xiangyu Wang, Yue Teng, Mapping the knowledge domains of building information modeling (BIM): a bibliometric approach, Autom. Constr. 84 (2017) 195–206, https://doi.org/10.1016/j.autcon.2017.09.011.

[5] Johnny Kwok Wai Wong, Janet Ge, Sean Xiangjian He, Digitisation in facilities management: a literature review and future research directions, Autom. Constr. 92 (2018) 312–326, https://doi.org/10.1016/j.autcon.2018.04.006.

[6] buildingSMART International, URL: https://www.buildingsmart.org/, 2008. Last accessed: 2022-07-26.

[7] International Organization for Standardization (ISO), ISO 16739-1:2018, Industry Foundation Classes (IFC) for data sharing in the construction and facility management industries — Part 1: Data schema, URL: https://www.iso.org/standard/70303.html, 2018. Last accessed: 2022-06-16.

[8] André Borrmann, Markus König, Christian Koch, Jakob Beetz, Building information modeling: Why? What? How?, in: Building Information Modeling: Technology Foundations and Industry Practice, 2018, ISBN 978-3-319-92862-3, pp. 1–24, https://doi.org/10.1007/978-3-319-92862-3_1.

[9] Junxiang Zhu, Wu Peng, BIM/GIS data integration from the perspective of information flow, Autom. Constr. 136 (2022) 104166, https://doi.org/10.1016/j.autcon.2022.104166.

[10] Ebrahim P. Karan, Javier Irizarry, Extending BIM interoperability to preconstruction operations using geospatial analyses and semantic web services, Autom. Constr. 53 (2015) 1–12, https://doi.org/10.1016/j.autcon.2015.02.012.

[11] Xiongwei Huang, Yongping Liu, Lizhen Huang, Erling Onstein, Christoph Merschbrock, BIM and IoT data fusion: the data process model perspective, Autom. Constr. 149 (2023) 104792, https://doi.org/10.1016/j.autcon.2023.104792.

[12] Shu Tang, Dennis R. Shelden, Charles M. Eastman, Pardis Pishdad-Bozorgi, Xinghua Gao, A review of building information modeling (BIM) and the internet of things (IoT) devices integration: present status and future trends, Autom. Constr. 101 (2019) 127–139, https://doi.org/10.1016/j.autcon.2019.01.020.

[13] Angel Ruiz-Zafra, Kawtar Benghazi, Manuel Noguera, IFC+: towards the integration of IoT into early stages of building design, Autom. Constr. 136 (2022) 104129, https://doi.org/10.1016/j.autcon.2022.104129.

[14] Gozde Basak Ozturk, Interoperability in building information modeling for AECO/FM industry, Autom. Constr. 113 (2020) 103122, https://doi.org/10.1016/j.autcon.2020.103122.

[15] Sandra Matarneh, Faris Elghaish, Farzad Pour Rahimian, Nashwan Dawood, David Edwards, Automated and interconnected facility management system: an open IFC cloud-based BIM solution, Autom. Constr. 143 (2022) 104569, https://doi.org/10.1016/j.autcon.2022.104569.

[16] Dean Allemang, James Hendler, Chapter 1 - what is the semantic web?, in: Semantic Web for the Working Ontologist Morgan Kaufmann, San Francisco, 2008, ISBN 978-0-12-373556-0, pp. 1–13, https://doi.org/10.1016/B978-0-12-373556-0.00001-0.

[17] Ahmed Gouda Mohamed, Mohamed Reda Abdallah, Mohamed Marzouk, Bim and semantic web-based maintenance information for existing buildings, Autom. Constr. 116 (2020) 103209, https://doi.org/10.1016/j.autcon.2020.103209.

[18] Rahel Kebede, Annika Moscati, He Tan, Peter Johansson, Integration of manufacturers' product data in BIM platforms using semantic web technologies, Autom. Constr. 144 (2022) 104630, https://doi.org/10.1016/j.autcon.2022.104630.

[19] Stefano Cursi, Letizia Martinelli, Nicolò Paraciani, Filippo Calcerano, Elena Gigliarelli, Linking external knowledge to heritage BIM, Autom. Constr. 141 (2022) 104444, https://doi.org/10.1016/j.autcon.2022.104444.

[20] World Wide Web Consortium (W3C), OWL Web Ontology Language Reference, URL: https://www.w3.org/TR/owl-ref/, 2004.

[21] World Wide Web Consortium (W3C), OWL 2 Web Ontology Language Document Overview, Second edition, 2013. URL: https://www.w3.org/TR/owl2-overview/. Last accessed: 2022-07-03.

[22] Calin Boje, Annie Guerriero, Sylvain Kubicki, Yacine Rezgui, Towards a semantic construction digital twin: directions for future research, Autom. Constr. 114 (2020) 103179, https://doi.org/10.1016/j.autcon.2020.103179.

[23] Manu Venugopal, Charles M. Eastman, Jochen Teizer, An ontology-based analysis of the industry foundation class schema for building information model exchanges, Adv. Eng. Inform. 29 (4) (2015) 940–957, https://doi.org/10.1016/j.aei.2015.09.006.

[24] Zhipeng Zhou, Yang Miang Goh, Li Jun Shen, Overview and analysis of ontology studies supporting development of the construction industry, J. Comput. Civ. Eng. 30 (6) (2016), https://doi.org/10.1061/(ASCE)CP.1943-5487.0000594, 04016026.

[25] Guoqian Ren, Haijiang Li, Song Liu, Jaliya Goonetillake, Ali Khudhair, Steven Arthur, Aligning BIM and ontology for information retrieve and reasoning in value for money assessment, Autom. Constr. 124 (2021) 103565, https://doi.org/10.1016/j.autcon.2021.103565.

[26] Pieter Pauwels, Walter Terkaj, EXPRESS to OWL for construction industry: towards a recommendable and usable ifcOWL ontology, Autom. Constr. 63 (2016) 100–133, https://doi.org/10.1016/j.autcon.2015.12.003.

[27] Pieter Pauwels, Sijie Zhang, Yong-Cheol Lee, Semantic web technologies in AEC industry: a literature overview, Autom. Constr. 73 (2017) 145–165, https://doi.org/10.1016/j.autcon.2016.10.003.

[28] Hu Shushan, Jiale Wang, Cathal Hoare, Yehong Li, Pieter Pauwels, James O'Donnell, Building energy performance assessment using linked data and cross-domain semantic reasoning, Autom. Constr. 124 (2021) 103580, https://doi.org/10.1016/j.autcon.2021.103580.

[29] Kris McGlinn, Rob Brennan, Christophe Debruyne, Alan Meehan, Lorraine McNerney, Eamonn Clinton, Philip Kelly, Declan O'Sullivan, Publishing authoritative geospatial data to support interlinking of building information models, Autom. Constr. 124 (2021) 103534, https://doi.org/10.1016/j.autcon.2020.103534.

[30] Pieter Pauwels, IFCtoRDF, URL: https://github.com/pipauwel/IFCtoRDF, 2017. Last accessed: 2022-07-26.

[31] B. Zhang, IfcSTEP-to-IfcOWL-converters, URL: https://github.com/BenzclyZhang/IfcSTEP-to-IfcOWL-converters, 2019.

[32] N. Salheb, K. Arroyo Ohori, J. Stoter, Automatic conversion of citygml to IFC, Int. Arch. Photogramm. Remote. Sens. Spat. Inf. Sci. XLIV-4/W1-2020 (2020) 127–134. https://isprs-archives.copernicus.org/articles/XLIV-4-W1-2020/127/2020/.

[33] Francesca Noardo, Lars Harrie, Ken Arroyo Ohori, Filip Biljecki, Claire Ellul, Thomas Krijnen, Helen Eriksson, Dogus Guler, Dean Hintz, Mojgan A. Jadidi, Maria Pla, Santi Sanchez, Ville-Pekka Soini, Rudi Stouffs, Jernej Tekavec, Jantien Stoter, Tools for BIM-GIS Integration (IFC Georeferencing and Conversions): results from the GeoBIM Benchmark 2019, ISPRS Int. J. Geo Inf. 9 (9) (2020) 502. https://www.mdpi.com/2220-9964/9/9/502.

[34] World Wide Web Consortium (W3C), RDF Primer, URL: https://www.w3.org/TR/rdf-primer/, 2004. Last accessed: 2022-07-03.

[35] Fajar J. Ekaputra, Xiashuo Lin, SHACL4P: SHACL constraints validation within Protégé ontology editor, in: 2016 International Conference on Data and Software Engineering (ICoDSE), 2016, pp. 1–6, https://doi.org/10.1109/ICODSE.2016.7936162.

[36] Julien Corman, Juan L. Reutter, Ognjen Savković, Semantics and validation of recursive SHACL, in: 17th International Semantic Web Conference (ISWC), 2018, pp. 318–336, https://doi.org/10.1007/978-3-030-00671-6_19.

[37] World Wide Web Consortium (W3C), Shapes Constraint Language (SHACL), URL: https://www.w3.org/TR/shacl/, 2017. Last accessed: 2022-07-03.

[38] Ranjith K. Soman, Modelling construction scheduling constraints using shapes constraint language (SHACL), in: Proceedings of the 2019 European Conference on Computing in Construction 1, 2019, pp. 351–358, https://doi.org/10.35490/EC3.2019.170.

[39] Jose Emilio Labra-Gayo, Herminio García-González, Daniel Fernández-Alvarez, Eric Prud'hommeaux, Challenges in RDF validation, in: Current Trends in Semantic Web Technologies: Theory and Practice, 2019, ISBN 978-3-030-06149-4, pp. 121–151, https://doi.org/10.1007/978-3-030-06149-4_6.

[40] Paolo Pareti, George Konstantinidis, A review of SHACL: from data validation to schema reasoning for RDF graphs, in: Reasoning Web. Declarative Artificial Intelligence: 17th International Summer School 2021, Leuven, Belgium, September 8–15, 2021, Tutorial Lectures, 2022, pp. 115–144, https://doi.org/10.1007/978-3-030-95481-9_6.

[41] Rui Zhu, Cogan Shimizu, Shirly Stephen, Zhou Lu, Ling Cai, Gengchen Mai, Krzysztof Janowicz, Mark Schildhauer, Pascal Hitzler, SOSA-SHACL: shapes constraint for the sensor, observation, sample, and actuator ontology, in: Proceedings of the 10th International Joint Conference on Knowledge Graphs, 2022, pp. 99–107, https://doi.org/10.1145/3502223.3502235.

[42] Ranjith K. Soman, Miguel Molina-Solana, Jennifer K. Whyte, Linked-data based constraint-checking (LDCC) to support look-ahead planning in construction, Autom. Constr. 120 (2020) 103369, https://doi.org/10.1016/j.autcon.2020.103369.

[43] Jose Emilio Labra Gayo, Eric Prud'hommeaux, Iovka Boneva, Dimitris Kontokostas, Validating RDF data, in: Synthesis Lectures on the Semantic Web: Theory and Technology 7, 2018, ISBN 978-3-031-79477-3, pp. 1–328, https://doi.org/10.1007/978-3-031-79478-0 (1).

[44] Walter Terkaj, Aleksandra Šojić, Ontology-based representation of IFC EXPRESS rules: an enhancement of the ifcOWL ontology, Autom. Constr. 57 (2015) 188–201, https://doi.org/10.1016/j.autcon.2015.04.010.

[45] Hans Schevers, Robin Drogemuller, Converting the industry foundation classes to the web ontology language, in: 2005 First International Conference on Semantics, Knowledge and Grid, 2005, p. 73, https://doi.org/10.1109/SKG.2005.59.

[46] Pieter Pauwels, Thomas Krijnen, Walter Terkaj, Jakob Beetz, Enhancing the ifcOWL ontology with an alternative representation for geometric data, Autom. Constr. 80 (2017) 77–94, https://doi.org/10.1016/j.autcon.2017.03.001.

[47] Dongming Guo, Erling Onstein, Angela Rosa, An improved approach for effective describing geometric data in ifcOWL through WKT high order expressions, in: Proceedings of the 7th International Conference on Geographical Information Systems Theory, Applications and Management - GISTAM, 2021, pp. 229–236, https://doi.org/10.5220/0010532302290236.

[48] W. Zhao, J.K. Liu, OWL/SWRL representation methodology for EXPRESS-driven product information model: part I. Implementation methodology, Comput. Ind. 59 (6) (2008) 580–589, https://doi.org/10.1016/j.compind.2008.02.002.
[49] Nick Bassiliades, A tool for transforming semantic web rule language to SPARQL infererecing notation, Int. J. Semant. Web Inf. Syst. 16 (1) (2020) 87–115, https://doi.org/10.4018/ijswis.2020010105.
[50] Pekka Aarnio, Valeriy Vyatkin, David Hästbacka, Context modeling with situation rules for industrial maintenance, in: 2016 IEEE 21st International Conference on Emerging Technologies and Factory Automation (ETFA), 2016, pp. 1–9, https://doi.org/10.1109/ETFA.2016.7733539.
[51] Ali Kücükavci, Mikki Seidenschnur, Kristoffer Negendahl, Christian Anker Hviid, Taking advantage of semantic web ontologies and shape constraints for Heating, Cooling and Ventilation Systems, in: E3S Web of Conferences 362, 2002, p. 7, https://doi.org/10.1051/e3sconf/202236204002.
[52] Adam Tamas Kovacs, Andras Micsik, Bim quality control based on requirement linked data, Int. J. Archit. Comput. 19 (3) (2021) 431–448, https://doi.org/10.1177/14780771211012175.
[53] Al-Hakam Hamdan, Jakob Taraben, Marcel Helmrich, Tobias Mansperger, Guido Morgenthal, Raimar J. Scherer, A semantic modeling approach for the automated detection and interpretation of structural damage, Autom. Constr. 128 (2021) 103739, https://doi.org/10.1016/j.autcon.2021.103739.
[54] Jeroen Werbrouck, Madhumitha Senthilvel, Jakob Beetz, Pieter Pauwels, A checking approach for distributed building data, in: 31. Forum Bauinformatik Proceedings, 2019, pp. 173–181, https://doi.org/10.14279/depositonce-8763.
[55] Markus Hagedorn, Philippand König, Rule-Based Semantic validation for standardized linked building models, in: Proceedings of the 18th International Conference on Computing in Civil and Building Engineering 98, 2021, pp. 772–787, https://doi.org/10.1007/978-3-030-51295-8_53.
[56] Madhumitha Senthilvel, Jakob Beetz, A visual programming approach for validating linked building data, in: European Group for Intelligent Computing in Engineering (EG-ICE) 2020 proceedings:International Workshop on Intelligent Computing in Engineering, 2020, pp. 403–411, https://doi.org/10.14279/DEPOSITONCE-9977.
[57] Andre Borrmann, Jakob Beetz, Christian Koch, T. Liebich, Sergej Muhic, Industry foundation classes: a standardized data model for the vendor-neutral exchange of digital building models, in: Building Information Modeling: Technology Foundations and Industry Practice, 2018, ISBN 978-3-319-92862-3, pp. 81–126, https://doi.org/10.1007/978-3-319-92862-3_5.
[58] International Organization for Standardization (ISO), ISO 10303-11:2004, – Industrial automation systems and integration – Product data representation and exchange – Part 11: Description methods: The EXPRESS language reference manual, URL: https://www.iso.org/standard/38047.html/, 2004.
[59] buildingSMART, Industry Foundation Classes Release 4.3.0.0 (ifc4x3), URL: https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/, 2022. Last accessed: 2022-07-26.
[60] Wikipedia, EXPRESS(data modeling language), URL: https://en.wikipedia.org/wiki/EXPRESS_(data_modeling_language), 2004. Last accessed: 2022-07-26.
[61] World Wide Web Consortium (W3C), SPARQL 1.1 Query Language, URL: https://www.w3.org/TR/sparql11-query/, 2013. Last accessed: 2022-05-10.